IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 18 December 2025, accepted 20 January 2026, date of publication 28 January 2026, date of current version 6 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3658743

TUTORIAL

The RISC-V FPGA (RVfpga) Teaching Package

DANIEL CHAVER[1], SARAH HARRIS[2], LUIS PINUEL[1], OLOF KINDGREN[3], ZUBAIR KAKAKHEL[4], CHRIS OWEN[5], ROY KRAVITZ[6], JOSE I. GOMEZ-PEREZ[1], FERNANDO CASTRO[1], KATZALIN OLCOZ[1], (Member, IEEE), JULIO VILLALBA-MORENO[7], ALEXANDER GRINSHPUN[8], FREDDY GABBAY[9], (Senior Member, IEEE), LUKE SEED[10], (Member, IEEE), RUI DUARTE[11], (Senior Member, IEEE), MANUEL LÓPEZ[12], ÓSCAR ALONSO[12], AND ROBERT OWEN[13]

[1]Computer Architecture and Automation Department, Complutense University of Madrid, 28040 Madrid, Spain
[2]Department of Electrical and Computer Engineering, University of Nevada, Las Vegas, NV 89005, USA
[3]Qamcom Research and Technology, 41285 Gothenburg, Sweden
[4]AZKY Tech Laboratories, LS8 2AL London, U.K.
[5]Cotronix, GL10 2NG Gloucestershire, U.K.
[6]ECE Department, Portland State University, Portland, OR 97201, USA
[7]Department of Computer Architecture, University of Málaga Complejo Tecnologico, Campus de Teatinos, 29071 Malaga, Spain
[8]Ruppin Academic Center, Technion – Israel Institute of Technology, The Hebrew University of Jerusalem, Jerusalem, Israel
[9]Institute of Electrical Engineering and Applied Physics, The Hebrew University, Jerusalem 9190401, Israel
[10]Department of Electronic and Electrical Engineering, The University of Sheffield, S10 2TN Sheffield, U.K.
[11]INESC-INOV, ISEL-IPL, 1959-007 Lisboa, Portugal
[12]University of Barcelona, 08028 Barcelona, Spain
[13]Essaimage Associates, SG14 3JW Hertford, U.K.

Corresponding author: Fernando Castro (fcastror@ucm.es)

This work was supported in part by Ministerio de Ciencia, Innovación y Universidades (MCIN)/Agencia Estatal de Investigación (AEI) under Grant PID2024-158311NB-I00, in part by "European Regional Development Fund (ERDF) a way of making Europe," in part by Imagination Technologies, in part by the University of Nevada, Las Vegas, in part by the Interuniversity Microelectronics Centre (Imec), Leuven, Belgium, and in part by the University of Nevada, Las Vegas (UNLV), University Libraries Open Article Fund.

ABSTRACT RISC-V is a free and open-standard ISA based on RISC principles, allowing anyone to design, manufacture, and sell RISC-V chips and software. Its flexibility and growing ecosystem have made it popular in research, education, and industry, increasing the need for educational materials. This paper provides an in-depth description of the RVfpga course, which offers a solid introduction to computer architecture using the RISC-V instruction set and FPGA technology. It focuses on providing hands-on experience with real-world RISC-V cores, the VeeR EH1 and EL2 cores, developed by Western Digital and hosted by ChipsAlliance. The course targets students and educators in computing-related fields, enabling them to integrate practical RISC-V knowledge into their curricula. The course materials, which include detailed labs, setup guides, and the full SoC source code in System Verilog, are available for free. Students learn to compile, debug, and run C and assembly programs, to interact with built-in peripherals, to extend the SoC, and to explore microarchitectural features.

INDEX TERMS Computer architecture, computer science education, education courses, microarchitecture.

I. INTRODUCTION

Risc-v is a free and open-standard instruction set architecture (ISA) introduced in 2010 at the University of California, Berkeley. Its design adheres to the principles of the reduced instruction set computer (RISC) philosophy, emphasizing simplicity and efficiency. Since its inception, RISC-V has gained widespread adoption across industry, academia, and research, thanks to its flexibility, scalability, modularity and absence of licensing fees, making it an attractive choice for a wide range of computing applications. The RISC-V architects established five main principles for its design:

- Compatibility: Software and hardware must work seamlessly across different implementations of the ISA.
- Feasibility: Its implementation must be feasible across various technology options, including FPGAs (field-programmable gate arrays), ASICs (application-specific integrated circuits), and emerging technologies.

The associate editor coordinating the review of this manuscript and approving it for publication was Alba Amato.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

- Efficiency: It must be efficient across various microarchitectural scenarios, including those using microcode or hardwired control, in-order or out-of-order pipelines, and various types of parallelism.
- Customization: It must be adaptable to specific tasks to achieve optimal performance without limitations imposed by the ISA itself.
- Stability: Its base instruction set must be stable and long-lasting, providing a common and robust framework for developers.

The ecosystem built around the RISC-V architecture, including the toolchain, processors and simulators, has been steadily developing since RISC-V's inception in 2010. Despite these developments, the ability to readily access, use, and work with the RISC-V architecture and open-source RISC-V systems still faces barriers, including lack of understanding the architecture and its extensions, the inability to understand and use RISC-V tools, and the difficulty in gathering and using all of the pieces needed to deploy, understand, and expand a commercial, non-trivial, RISC-V core and SoC with its accompanying development environment and tools. While some courses address some of these barriers, the RISC-V FPGA (RVfpga) courses (the RVfpga in Computer Architecture [1] and the RVfpga-SoC in SoC design [2]) aim to address all of these barriers by showing the complete path from system setup to using, programming, understanding, and expanding a RISC-V core and SoC.

In RVfpga we provide two SoCs, one based on the VeeR EH1 core [3] and one based on the VeeR EL2 core [4]. These SoCs can be deployed on different FPGA boards, from low-cost ones such as Basys3 (Artix-7 from Xilinx) [5] or Boolean (Spartan-7) [6] boards to more expensive ones such as the Nexys A7 board (Artix-7 from Xilinx) [7]. The RVfpga SoC can also run on different simulation tools: RVfpga-ViDBo, RVfpga-Pipeline, RVfpga-Trace, and RVfpga-Whisper.

While the RVfpga courses have been previously introduced in earlier publications [8], [9], this paper makes four main contributions. First, it provides an up-to-date description of the RVfpga ecosystem, including commercial-grade RISC-V SoCs, simulators, and FPGA deployments (Sections II-A to II-C). Second, it analyzes how RVfpga aligns with the most recent IEEE/ACM/AAAI computing curriculum guidelines (Section II-D). Third, it consolidates educational experiences spanning undergraduate, graduate, and professional contexts, including their adaptation to a MOOC on the edX platform [10] (Section III). Finally, it discusses in detail several related RISC-V–based educational platforms (Section IV).

II. THE RVfpga TEACHING PACKAGE

A. THE RVfpga SYSTEM

1) SOFTWARE AND HARDWARE USED IN RVfpga

Table 1 summarizes the software and hardware used in the RVfpga courses. All materials are provided in the RVfpga packages or are available for free download, except for the optional FPGA board.

TABLE 1. Software and hardware used in RVfpga.

Cores	
VeeR EH1	9-stage, superscalar RISC-V core
VeeR EL2	4-stage, scalar RISC-V core
SoCs	
VeeRwolfX EH1	SoC based on the VeeR EH1 core
VeeRwolfX EL2	SoC based on the VeeR EL2 core
FPGA boards (optional)	
Nexys A7 / Nexys 4 DDR	FPGA boards from Digilent, Inc.
Basys 3 (only EL2)	Lower-cost FPGA board from Digilent, Inc.; only the EL2-based SoC fits on this board
Boolean (only EL2)	Lower-cost FPGA board from Real Digital; only the EL2-based SoC fits on this board.
Simulation tools	
RVfpga-ViDBo (based on Verilator)	Web-based virtual board that simulates the RVfpga System running a program that interacts with FPGA peripherals
RVfpga-Pipeline (based on Verilator)	Tool that simulates the RVfpga System running a program, showing the instructions executing through the VeeR EH1/EL2 pipeline
RVfpga-Trace (based on Verilator)	Tool that simulates the RVfpga System running a program, showing a waveform of the system's internal signals during execution.
Whisper	RISC-V instruction set simulator (ISS)
SDK (software development kit)	
Catapult SDK	Open-source RISC-V SDK developed by Imagination Technologies.
Platform IO	Open-source SDK that is a Visual Studio Code extension.
Synthesis tool (optional)	
Vivado 2022.2 WebPACK	Vivado synthesizes the Verilog RVfpga SoC and generates the bitstream for running on the FPGA.
Operating Systems	
Linux	Supports all tools.
Windows and macOS	Most tools are supported but are more challenging to install and run more slowly than in Linux.
Virtual machine	
Ubuntu 22.04	Virtual machine (VM) with all tools pre-installed. Works with VMware or VirtualBox.

TABLE 2. VeeR cores.

Name	ISA	Pipeline	Threads	CoreMark/MHz
VeeR EH1	RV32IMC	9 dual-issue	Single	4.9
VeeR EH2	RV32IMAC	9 dual-issue	Dual	6.3
VeeR EL2	RV32IMC	4 single-issue	Single	3.6

2) RVfpga CPUs

Western Digital developed three RISC-V cores in recent years. These VeeR EH1, EH2, and EL2 cores (main features in Table 2), formerly called SweRV cores, have an Apache 2.0 license and are currently hosted by ChipsAlliance.

In RVfpga we use the VeeR EH1 [3] and VeeR EL2 [4] cores. VeeR EH1 is a 32-bit RV32IMC CPU core operating in machine mode (M-mode) only. It supports RISC-V's I, M, C, and Zcsr extensions and features a superscalar, dual-issue 9-stage pipeline with four ALUs, separate pipelines for load/store and multiplication, and an out-of-pipeline 34-cycle divider. It has stall points in Fetch1, Align, Decode, and Commit stages. VeeR EL2 is a smaller 32-bit RV32IMC core with a scalar 4-stage pipeline, supporting I, M, C, Zcsr, and some bit manipulation (Zb*) extensions. It includes integer, load/store, and multiplier pipelines, with an out-of-pipeline divider and stall points in the Fetch and Decode stages.

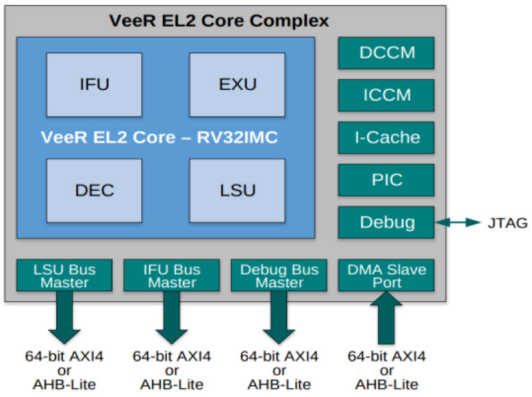


FIGURE 1. The VeeR EL2 Core Complex (figure from [4]).

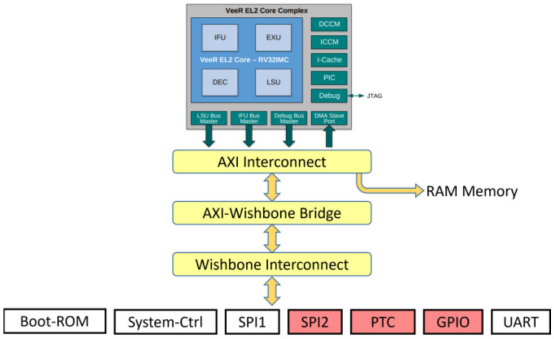


FIGURE 2. The VeeRwolfX SoC.

ChipsAlliance provides a Core Complex (Fig. 1) for both cores, which add closely-coupled instruction and data memories (ICCM and DCCM), a 4-way instruction cache, a programmable interrupt controller (PIC), four system bus interfaces, and a core debug unit compliant with the RISC-V Debug specification.

3) RVfpga SoCs

The SoC used in RVfpga, called VeeRwolfX (VeeRwolf eXtended), is an extended version of VeeRwolf [11], which is built around the VeeR EH1/EL2 Core Complex. Fig. 2 illustrates the VeeRwolfX SoC for the VeeR EL2 version; the EH1 SoC swaps EL2 with the EH1 Core Complex.

The VeeRwolf SoC includes a boot ROM, UART, system controller, and an SPI controller (elements in white in Fig. 2). The VeeRwolfX SoC adds functionality (in red in Fig. 2), such as another SPI controller (SPI2), a GPIO (general-purpose input/output) controller, a PTC (PWM/timer/counter) module, and a controller for interfacing with the 8-digit 7-segment displays (as part of the system controller). The VeeRwolfX SoC does not include a memory controller, but it reserves the first 128 MiB of memory and exposes the AXI bus, so that the user can access RAM memory.

4) RVfpga SIMULATORS AND FPGA BOARDS

We provide both simulation and optional hardware methods for using and analyzing the RVfpga core, SoC, and its peripherals. Fig. 3 shows a diagram of these tools using the EL2 SoC and a Nexys A7 FPGA board. All the tools share a common back-end, the VeeRwolfX SoC code, but have different front-ends. We refer to the SoC's source code, Verilog and SystemVerilog code, as the RTL or Verilog code. The videos in [12] and [13] show the various simulators and boards executing sample programs. More examples are available in the RVfpga MOOC [10] and in the whole 2022 RVfpga Tutorial [14].

a: EXECUTION ON THE PHYSICAL BOARD

Recall that running the RVfpga SoC in hardware is optional, as all activities can be completed in simulation instead. To use the VeeRwolfX SoC in hardware on an FPGA board, we provide the default configurations with the packages; users could generate a new bitstream using Vivado as shown in Fig. 3. RVfpga supports three FPGA boards: the Nexys A7 ($349, academic cost $262), Basys 3 ($165, academic cost $124), and Boolean ($74) boards.

b: EXECUTION IN SIMULATION

RVfpga includes three SoC simulation tools based on Verilator [15], as shown in Fig. 3: RVfpga-ViDBo, RVfpga-Pipeline, and RVfpga-Trace. These tools, in addition to RVfpga-Whisper, can be used to complete all the labs without needing a physical board; thus, the RVfpga course can be completed at no cost. The back-end of each of these SoC simulators is Verilator, which transforms the Verilog SoC into C++ code; the user then compiles and executes that code to run a RISC-V program in simulation. Each SoC simulation tool then uses those simulation results. The simulation binaries for the default configurations are provided with the RVfpga package, but users can generate new binaries using Verilator. We next describe each of these simulation tools:

1) **RVfpga-ViDBo**: This tool uses a Virtual Development Board [16] to perform a simulation of the RVfpga System and communicate with the peripherals on the simulated board. The virtual board (front-end) visualizes the simulation (RISC-V program running on the RVfpga System) on a virtual FPGA board viewable in a web browser (see Fig. 3 and Fig. 5). The back-end supports either VeeR core (EL2 and EH1), and the front-end supports all FPGA boards (Nexys A7, Basys 3, and Boolean), each one including a different set of peripherals. The most complete configuration, the virtual board simulates 16 switches and 16 LEDs, the UART, 5 pushbuttons, the 8-digit 7-segment display, and a tricolor LED.

2) **RVfpga-Pipeline**: This simulator shows the instructions of a RISC-V program flowing through the VeeR EH1 or EL2 pipeline. The front-end of this simulator uses GTK, a free and open-source cross-platform widget, to show the instructions and signals in the pipeline stages using the simulated program produced by Verilator (see Fig. 3). RVfpga-Pipeline shows the details in a given instruction stage, including the assembly instruction executing and the values of processor signals

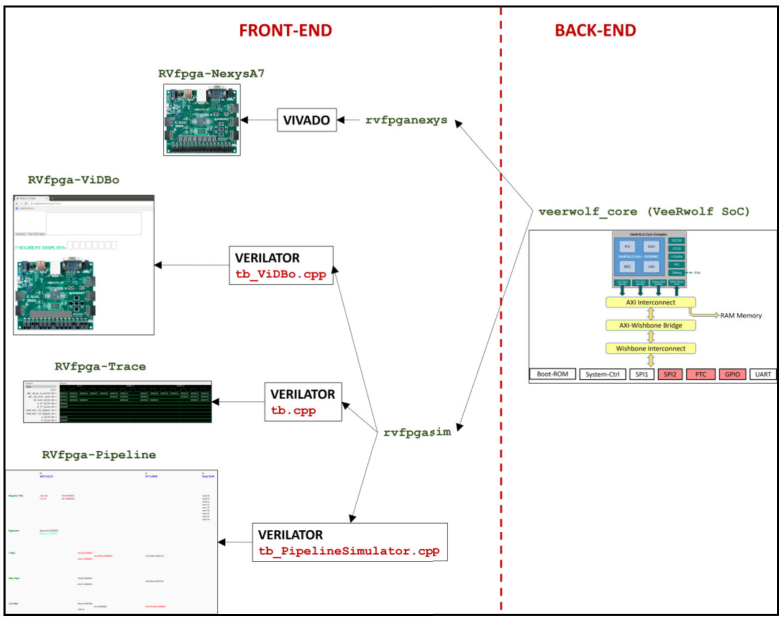


FIGURE 3. RVfpga tools: front-end and back-end.

involved in the instruction's execution, such as the register file read input/output signals, the ALU input/output signals, the register file write input signals, etc.

3) **RVfpga-Trace**: This simulator generates a trace of the internal signals of the SoC while executing a given program and then visualizes the waveform of this trace (see Fig. 3) using GTKWave. Any of the SoC signals can be selected and visualized in the waveform.

B. MATERIALS INCLUDED IN RVfpga

The RVfpga course provides a Getting Started Guide, 20 labs that include theoretical explanations, examples, and exercises, the SoC Verilog source code, solutions to all the exercises, videos, etc. We next summarize these materials.

1) GETTING STARTED GUIDE

The Getting Started Guide (GSG) is the introductory document of the RVfpga course. It is a comprehensive document that, among other things, provides: an overview of the RISC-V architecture and exhaustive analysis of the RVfpga System; in-depth instructions for the three supported operating systems (Linux, Windows, and macOS) on how to install all the tools used in the RVfpga course; detailed instructions about how to write, simulate, and run programs on the RVfpga SoC by using example programs; and a set of appendices that explain, among other things, how to use the native RISC-V toolchain and OpenOCD from the command-line, in place of PlatformIO or Catapult SDK.

2) RVfpga LABS

The RVfpga Labs are a set of labs that aim to help students understand computer architecture concepts. Apart from the software described above, extensive lab scritps are provided, which include theoretical explanations and instructions, as well as many examples, exercises and solutions. The labs are divided into two main parts. Part 1 shows how to program the RVfpga SoC in C and RISC-V assembly, how to interact with the various peripherals –using both program-driven and interrupt-driven I/O– and how to extend the RVfpga SoC to include additional devices. Part 2 of the labs focuses on the microarchitecture of the RISC-V VeeR cores and the memory system.

a: PART 1 (RISC-V ARCHITECTURE AND INPUT/OUTPUT)

Labs 1-4 show how to run C and RISC-V assembly programs on the board and in simulation. Lab 1 and Lab 2 show how to create simple C and RISC-V assembly projects that can be run on the RVfpga System. A tutorial on how to create and run a program is first provided; then several exercises are proposed for students to practice writing their own programs. These labs also explain how to print to the terminal using the UART. Lab 3 shows how to write and use functions in C and RISC-V assembly programs, describes the RISC-V ABI (application binary interface), and then provides exercises on writing C and RISC-V assembly programs that use functions and library calls. Lab 4 shows how to build a complete RISC-V programming project that transforms an image from RGB (color) to gray-scale. The project includes source files in C or assembly. At the end of the lab, several exercises are provided to add functionality to the image processing algorithms. The exercises also guide students in translating functions from C to assembly and vice versa, which requires a deep understanding of the RISC-V ABI.

Labs 1-4 can be completed on the board or in simulation, with the only difference being that the simulator may run more slowly. Specifically, the Whisper ISS simulator can

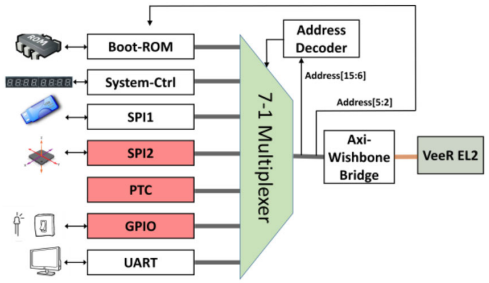


FIGURE 4. Connection of peripherals in the EL2-based SoC.

be used for all exercises except for those where peripherals (the LEDs, switches, etc.) are used, in which case the RVfpga-ViDBo simulator can be used instead.

Labs 5-10 show how to use and expand the RVfpga input/output (I/O) system to enable the RISC-V processor to interact with peripheral devices. These labs introduce common peripherals such as pushbuttons, 7-segment displays, and serial ports; they show how to use both polling and interrupts to access memory-mapped I/O peripherals; and they show how to expand the RVfpga system to use new peripherals.

Fig. 4 illustrates how the peripherals (Boot-ROM, System-Controller, SPI1 and SPI2, PTC, GPIO, and UART) are connected in the VeeR EL2-based SoC. Fig. 4 is a detailed view of the Wishbone Interconnect from Fig. 2. The 7:1 multiplexer selects one peripheral based on bits [15:6] of the address supplied by the VeeR EL2 core; address bits [5:2] are provided to the controller to access the specific memory-mapped register. Remember that the core uses an AXI bus (shown by the orange line in the figure), whereas the peripherals use a Wishbone bus (shown as a grey line in the figure). Thus, a Wishbone-to-AXI bridge is used to communicate between the core and the peripherals.

Lab 5 shows how to optionally create a Vivado project that targets the RVfpga SoCs to the FPGA boards. Lab 6 describes the main features of a general-purpose I/O (GPIO) system and then focuses on the GPIO controller used in the VeeRwolfX SoC by first analyzing its high-level specification and introducing fundamental exercises, where the user develops C and RISC-V assembly programs that read/write the memory-mapped GPIO registers to control the switches and LEDs. Then, the lab analyzes the low-level implementation of the GPIO peripheral and introduces advanced exercises where the user adds onboard pushbuttons to the RVfpga system by adding a new GPIO peripheral to the SoC and then writing new C and assembly programs. These advanced exercises require users to first modify the SoC's hardware to connect the new GPIO controller with the VeeR core and with the board device (pushbuttons), debug the SoC modifications using RVfpga-Trace, generate a new bitstream or simulator binary, and test programs in the board or on RVfpga-ViDBo.

Using a similar structure as Lab 6, Lab 7 introduces another common peripheral, the 7-segment displays, starting with

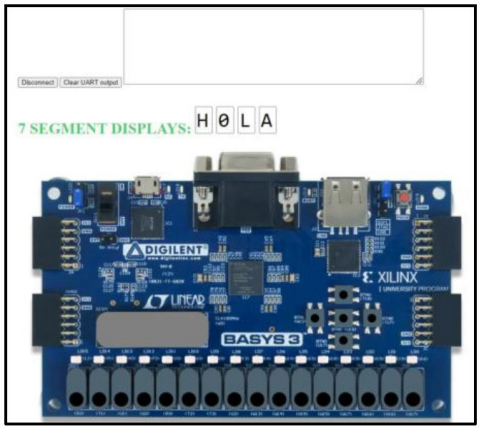


FIGURE 5. Modification of the 7-segment displays.

the high-level specification and basic exercises and then continuing onto describing the low-level implementation and advanced exercises to extend the peripheral. Specifically, users are guided to modify the 7-segment display controller to show any segment combination, not just the hexadecimal digits of the default controller. Fig. 5 shows a program executing in the extended RVfpga-ViDBo simulator for the Basys 3 board, which writes the Spanish word "HOLA" (which means "hello" in English) on the modified 7-segment display (the program would show the same results if executed on the physical FPGA board).

Using a similar structure as Labs 6 and 7, Lab 8 introduces another common peripheral, the PTC (PWM/timer/counter), starting with the high-level specification and basic exercises and continuing onto the low-level implementation and advanced exercises that extend the peripheral. Specifically, the user is guided to modify the PTC so that it controls the tri-color LEDs using PWM.

Lab 9 introduces the concept of interrupts and shows how to use them in several examples and programs. The lab first describes the differences between programmed I/O and interrupt-driven I/O. Then, it explains the operation of the RVfpga system's interrupt controller and describes how to configure and use external interrupts using Western Digital's (WD's) libraries. Finally, the lab shows example programs and provides exercises for using and extending the RVfpga System's hardware interrupts.

To conclude part 1, Lab 10 describes how serial buses work and the main features of the SPI bus, one of the most typical serial buses currently used. The lab then analyzes and uses the FPGA board's SPI accelerometer in several exercises. This lab is not available for the Basys 3 or Boolean board, as these boards do not include SPI/I2C serial peripherals. However, this lab could be completed on these boards by extending them using external PMOD serial devices.

Labs 5-10 can be interchangeably run on the board or on the RVfpga-ViDBo simulator, with the only difference being that the simulator may run more slowly. Even those exercises where the SoC must be extended can be performed either on the board or in simulation (mainly using

RVfpga-ViDBo, which includes all the peripherals that we use in these exercises except for the SPI accelerometer). In those advanced exercises, a new bitstream (for the board) or a new simulation binary (for the RVfpga-ViDBo simulation) must be generated by following the instructions provided in Lab 5 and in the GSG, respectively. The RVfpga-Trace simulator is also required for debugging and verifying the changes performed to the Verilog code.

The videos from [10], [14], [16], and [17] provide example programs that run on the Nexys A7 FPGA board and interact with the various peripherals, both for the basic and advanced exercises (those where the SoC is extended) in Labs 6-9.

b: PART 2 (MICROARCHITECTURE AND MEMORY SYSTEM)

Labs 11-20 focus on microarchitecture and memory hierarchy. Fig. 6 illustrates the VeeR EH1 pipeline, as provided in the labs, for analyzing microarchitectural features. The three initial pipeline stages (Fetch1, Fetch2, and Align) appear at the top, while the six final stages (Decode, EX1, EX2, EX3, Commit, and Writeback) appear at the bottom. The execution stages (EX1-EX3) support four 3-stage pipelines (I0 and I1 integer pipelines, L/S: load/store pipeline, and the Mult: multiply pipeline) as well as a non-pipelined divider. Lab 11 provides these figures, as well as the equivalent EL2 pipeline figures, all accompanied by detailed descriptions.

The labs in part 2 show how to understand the RISC-V pipeline and the RVfpga SoC memory system as well as how to use or add features in the core, including additional instructions. These labs also show how to measure performance using the hardware counters. All labs begin with a theoretical explanation of the lab's concepts and then illustrate those concepts with the execution of an example program. The labs also provide exercises to deepen understanding of and experience with the described concepts.

Note that all Verilator-based simulators are essential in using this group of labs (Labs 11-20). RVfpga-Trace is used in most labs for analyzing internal signals of the RVfpga SoC and core to illustrate various situations (i.e., hazards, superscalar execution, etc.). To reinforce concepts, a basic program and its waveform are analyzed, and the user is then asked to include more signals and inspect other programs in a similar way. RVfpga-Pipeline offers a complementary (though less flexible) view of the core. Finally, RVfpga-ViDBo can be useful when a board is not available, as the exercises related to performance evaluation must be executed on a physical (FPGA) or virtual (RVfpga-ViDBo) board.

Lab 11 is an introductory lab that shows how to analyze a VeeR core. It also presents concepts, source code organization, functionality, and applications that are used in all the labs in part 2. It starts by describing the RTL (Verilog code) organization of each pipeline stage, with detailed descriptions and figures of each stage as well as an RVfpga-Trace simulation of a simple program to clarify the pipeline organization. Moreover, it includes some tasks and exercises where the user inspects some pipeline stages and their associated signals.

Lab 11 also explains how to use performance counters by giving a guided example that describes how to set up the counters (using Western Digital's Processor Support Package [18]) and how to use them in an example program. Performance counters are also used in most of the remaining labs to evaluate program metrics such as number of instructions retired, number of instruction cache misses, etc. This lab also shows how to change configuration parameters in the VeeR cores (such as the size of the instruction cache or scratchpad memory, the branch predictor configuration, superscalar width, etc.) and how to evaluate the impact of those changes.

Finally, this lab describes and recommends using Sigasi Studio for working with large Verilog/SystemVerilog projects such as the RVfpga SoC and describes the organization and function of the SoC's submodules.

Lab 12 describes and analyzes the flow of arithmetic and logical instructions through the VeeR EH1/EL2 pipeline stages and then provides exercises where users perform similar analyses for other RISC-V arithmetic and logical instructions (such as: and, or, xor, srl, addi, slli, etc.), as well as other exercises adopted from the two main reference books ([19], [20]) and adapted to RVfpga. Fig. 7 illustrates one of the example simulations included in the lab for analyzing the execution of an `add t3, t3, t4` instruction through the VeeR EH1 pipeline, showing the most relevant signals inside the Decode, EX1, and WB pipeline stages. As a complement to the waveform, Lab 12 also provides an illustration of the EH1 pipeline units involved in the instruction execution (Fig. 8) from cycle `i` to cycle `i+5`. As shown in the figures, the instruction (`add t3, t3, t4`, which is $0\times01DE0E33$ in machine code) is in the Decode stage in cycle `i`, where the register file is read (a=7 and b=1); then it progresses to the EX1 stage in cycle `i+1`, where the addition is computed in the ALU (out=8); and finally, in the Writeback (WB) stage (cycle `i+5`), the instruction writes register x28 with the result of the addition: waddr0=28 (write address/register number), wen0=1 (write enable), wd0=8 (write data).

Lab 13 starts by examining the pipeline stages used to execute load/store operations when reading a low-latency memory location –that is, one that does not stall the processor–. The lab then examines the execution of store instructions. Finally, the lab repeats the analysis when accessing a higher-latency main memory.

Labs 14-16 explore pipeline hazards. It illustrates several structural hazards that can occur in the VeeR processor. Each situation is resolved in a different way, resulting in a different performance-cost trade-off. Lab 15 analyzes how read-after-write (RAW) data hazards are resolved in the VeeR processor and then describes tasks and exercises to further explore and understand these hazards. Lab 16 completes the analysis of hazards by describing control hazards and analyzing branch instructions and the branch predictor available in the VeeR core. As mentioned, most of the part 2 labs (Labs 11-20) use the RVfpga-Pipeline simulation tool for inspecting programs.

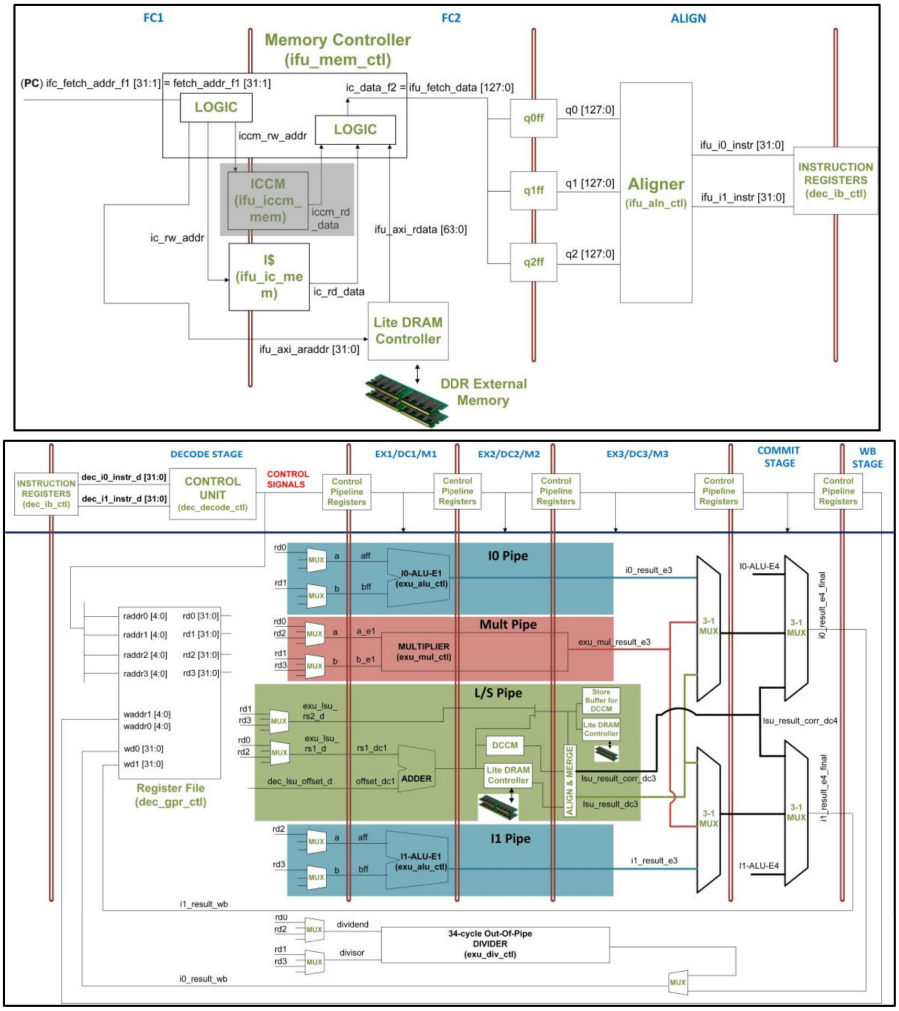


FIGURE 6. VeeR EH1 pipeline: the three initial stages are shown on top, and the six final stages are shown at the bottom.

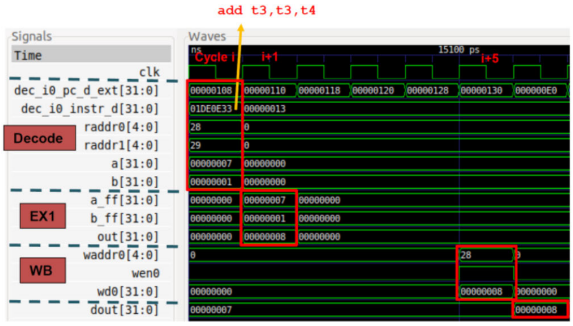


FIGURE 7. Execution of an add t3, t3, t4 instruction.

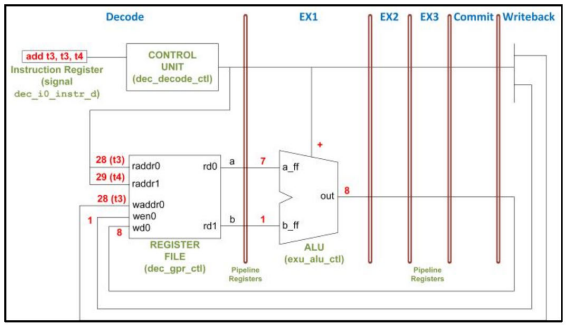


FIGURE 8. VeeR EH1 pipeline hardware involved in the execution of add t3, t3, t4.

While Labs 11-16 use only one processor way in most cases, Lab 17 describes 2-way superscalar processors, such as VeeR EH1. This lab is not available for the EL2-based SoC because the EL2 is a single-issue core. Lab 18 shows how to add new instructions to the VeeR core and then verify and test them. The lab does this by showing how to expand the core to support three floating-point instructions included in the RISC-V single-precision floating-point (Zfinx) extension: fadd.s, fmul.s, and fdiv.s. The lab then proposes exercises where the user repeats this process on their own, debugs the implementation using RVfpga-Trace, verifies it using the guidance of [17], and then tests the new instructions

on the board or in RVfpga-ViDBo. The user tests the new floating-point instructions by writing a program to implement the Bisection Method using both the new floating-point instructions and using floating point emulation. The user then compares the two alternatives using the performance counters described in Lab 11. This allows students to understand the complete path from the ISA specification to its implementation and evaluation in real applications.

Labs 19 and 20 focus on the low-latency memories available in the processor: the instruction cache (I$) and the closely-coupled instruction and data memories (ICCM and DCCM). Specifically, Lab 19 delves into the operation and management of the instruction cache (I$) available in the RVfpga System. Lab 20 analyzes the ICCM and DCCM and benchmarks the VeeR processors using two common embedded benchmarks (CoreMark and Dhrystone), which use the hardware counters described in Lab 11.

A user or instructor may choose to only complete a subset of the labs, depending on the aim and depth of the course:

- Programming in C and RISC-V assembly: Labs 1-4.
- Building the RVfpga SoC in Vivado: Lab 5.
- The input/output system: Labs 6-10.
- Pipelining: Labs 11-15 and first part of 16 (branch/jump instructions).
- Advanced microarchitecture and memory organization: Labs 13, second part of 16 (branch predictor), and 17-20.

3) OTHER MATERIALS INCLUDED IN RVfpga

In addition to the labs, the RVfpga materials include exercise solutions and source code. Solutions are provided for both the software (C and RISC-V assembly programs) and hardware (the Verilog SoC extensions), and these are contained in a single directory that professors may remove before sharing the RVfpga package with students.

All the Verilog source code for the RVfpga core and SoC are provided as well as the source code needed to target the RVfpga System to the boards (Basys 3, Boolean, and Nexys A7 for EL2; and only Nexys A7 for EH1) and simulators (RVfpga-ViDBo, RVfpga-Pipeline, and RVfpga-Trace). Using these sources, the user can modify the core (for example, to modify its configuration as shown in Labs 11, 16, and 20, or to add new instructions as in Lab 18) and the SoC (for example, including more peripherals as shown in Labs 6-10), and then generate a new simulator binary using Verilator (as explained in the GSG) or optionally generate a new bitstream using Vivado (as described in Lab 5). Precompiled bitstreams and simulators for the baseline core/SoC configuration are also included in the packages. The sources for all figures included in the documents are also provided in case users want to use or modify them.

The Platform Support Package (PSP) for the VeeR cores and the Board Support Package (BSP) for the different boards supported are also included as part of RVfpga. These packages, which can also be obtained with the WD RISC-V Firmware package [18], help users configure the RVfpga System and use interrupts or performance counters.

Finally, although this is not part of the RVfpga packages, the IUP website provides a forum where users can post questions and comments.

C. MATERIALS INCLUDED IN RVfpga-SoC

A second course, RVfpga-SoC, is a follow-on course to RVfpga and is provided as an independent package [2]. RVfpga-SoC covers SoC design and includes five labs. Lab 1 gives an overview of the course and shows how to create the RVfpga SoC from scratch by connecting modules including the VeeR EH1 core, interconnect, and peripherals. Lab 2 shows how to run programs on the RVfpga SoC built in Lab 1. Users may run the programs either in simulation or in hardware. Lab 3 introduces FuseSoC [21], an open-source tool for building systems, and compares the FuseSoC approach to the block design approach from Lab 1. Lab 4 shows how to build, run, and use Zephyr, an open-source real-time operating system (RTOS), on the RVfpga SoC. Finally, Lab 5 shows how to build a Tensorflow Lite project for Zephyr and then run that program on the RVfpga SoC.

D. COMPUTING CURRICULUM GUIDELINES IN RVfpga

RVfpga covers the topics in the latest ACM and IEEE computing curriculum guidelines for the Architecture and Organization (AR) area.

Several successive curricular guidelines for computing studies have been published over the years as the discipline of Architecture and Organization has continued to evolve. The Computer Science Curricula 2023 Final Report [22], also referred to as CS2023, is the latest version of these types of guidelines, produced by a joint task force of the ACM, IEEE Computer Society, and AAAI. Another older but wider report is ''Computing Curricula 2020: Paradigms for Global Computing Education'' [23], which, in addition to Computer Science, analyzes other computing areas such as Computer Engineering or Software Engineering.

The CS2023 knowledge model consists of 17 knowledge areas. Among these, the one that RVfpga seeks to cover is the AR area, described in detail in a dedicated section of the report (pages 101-111 of [22]). That section states in its preamble that, as the shift from sequential to parallel processing occurs, a deeper understanding of the underlying architecture is necessary, which can no longer be viewed as a closed box where principles from one architecture can be applied to another one. Instead, programmers should look inside the closed box and use specific components to enhance system performance and energy efficiency. The AR knowledge area aims to develop a deeper understanding of the hardware environments upon which almost all computing is based, and the relevant interfaces provided to higher software layers. The topics in this knowledge area will benefit students by enabling them to appreciate the fundamental architectural principles of modern computer systems, including the

challenge of harnessing parallelism to sustain performance and energy improvements into the future, and will help students depart from the closed box approach and become more aware of the underlying computer system and the efficiencies specific architectures can achieve.

Specifically, the AR area includes the following knowledge units, which are fully or partially covered in RVfpga:

- Digital Logic and Digital Systems: This unit includes topics such as FPGAs, Hardware Description Languages (HDLs) and SoC design flow, which are exhaustively covered in the RVfpga packages. For example, the RVfpga-SoC course covers SoC design flow, whereas labs 6 to 10 of RVfpga provide exercises where the student must understand and modify the I/O controllers, which are coded in Verilog/SystemVerilog, and run various programs on the RVfpga SoC on an FPGA board.
- Machine-Level Data Representation: This unit includes topics such as number bases and numeric data representations such as fixed-point and floating-point, which are discussed in RVfpga. Although these are traversal topics in RVfpga, as an example, Lab 18 includes a comprehensive exercise where the student must add a floating-point unit (FPU) to the VeeR core and then program, execute and compare the performance of different floating-point algorithms.
- Assembly Level Machine Organization: Topics such as the ISA (e.g., x86, ARM and RISC-V) and subroutine call and return mechanisms are included in this unit, and these topics are exhaustively covered in RVfpga. For example, RVfpga labs 2 to 4 describe the RISC-V ISA and provide numerous examples and exercises for the students to practice these concepts.
- Memory Hierarchy: This unit includes topics such as main memory organization and cache memories, which are covered in RVfpga. For example, Lab 19 describes the Instruction Cache used in the VeeR cores.
- Interfacing and Communication: I/O fundamentals (such as programmed I/O vs. interrupt-driven I/O), interrupt structures (vectored and prioritized), I/O devices and fundamentals of buses are included in this unit, which are covered in RVfpga. For example, RVfpga Lab 9 focuses on the use of interrupts in the RVfpga SoC and the differences between programmed I/O vs. interrupt-driven I/O.
- Functional Organization: This unit includes topics such as implementation of simple datapaths, instruction pipelining and hazard detection, which are exhaustively covered in RVfpga. For example, labs 11-13 analyze pipelined execution in the VeeR core and labs 14-16 analyze the different types of hazards: structural, data and control.
- Performance and Energy Efficiency: This unit includes topics such as performance, power consumption, memory evaluation, branch prediction, speculative execution, hardware support for multithreading, etc. Some of the topics covered in this unit are included in RVfpga; for example, Lab 20 evaluates performance using performance counters and two well-known benchmarks (Coremark and Dhrystone), and Lab 16 analyzes branch prediction.

Note that the remaining units in this knowledge area (Heterogeneous Architectures, Secure Processor Architectures and Quantum Architectures) are not covered in the current RVfpga packages. However, some of them could be included by extending the existing packages. For example, we could add support for VeeR EH2, a dual-threaded core, which would allow us to include labs that analyze the hardware support for multithreading topics mentioned in the Heterogeneous Architectures unit. We could also add a lab using the RISC-V bitmanip extension –implemented in VeeR– in cryptographic applications, covering the Cryptographic Acceleration with Hardware topic mentioned in the Secure Processor Architectures unit. The use of the Caliptra Open-Source Root of Trust Project [24] could also be analyzed to help cover this unit.

As for the other report mentioned above (‘‘Computing Curricula 2020: Paradigms for Global Computing Education’’ [23]), it refers not only to Computer Science but to six more computing areas: Computer Engineering, Cybersecurity, Information Systems, Information Technology, Software Engineering, and Data Science. This report defines thirty-four abbreviated knowledge areas for computing students partitioned into an ordered sequence of six categories, one of which is devoted to Hardware, which includes knowledge areas covered in RVfpga, such as Architecture and Organization and Digital Design; a second category is devoted to Systems Architecture and Infrastructure, which includes knowledge areas also covered in RVfpga, such as embedded systems. Example topics included in these knowledge areas are control and datapaths, programmable logic, processor organization, memory system organization and architecture, and I/O interfacing, all of which are exhaustively analyzed and discussed in RVfpga.

III. EXPERIENCES USING RVfpga

In this section, several professors from different universities around the world describe their experience using RVfpga in their formal bachelor/master's degree teaching activities (Sections III-A and III-B). Then, we describe other experiences related to RVfpga (Sections III-C III-D and III-E), including workshops, MOOCs, and research.

A. BACHELOR AND MASTER' DEGREE COURSES

Several universities use RVfpga in their Computer Architecture courses. Next we describe some of their experiences.

1) INTEGRATED SYSTEMS ARCHITECTURE (ISA) AT UCM

This is a fourth-year course in the Electronics and Communication Engineering bachelor's degree at UCM. The students of this course are in the last year of their

TABLE 3. Modules and labs included in ISA at UCM.

Module 1: RISC-V ISA	
Lab 1	RISC-V ISA review
Module 2: Processor Design	
Lab 2	Introduction to the 5-stage core by Harris&Harris and to the VeeR EH1 core
Lab 3	Performance of the 5-stage and VeeR EH1 cores
Lab 4	Execution of basic instructions on the VeeR EH1 core
Module 3: Memory Hierarchy	
Lab 5	The cache on the Ripes simulator and on the VeeR EH1 core
Lab 6	The scratchpad on the VeeR EH1 core
Module 4: Input/Output	
Lab 7	High-level I/O on the SoC
Lab 8	Low-level I/O on the SoC
Module 5: SoC Design	
Lab 9	RVfpga-SoC

bachelor's degree and have a background in digital design, hardware description languages (HDLs), computer architecture/organization, C programming and operating systems; thus, they have enough skills to complete most of the RVfpga and RVfpga-SoC labs, and only lack of time precludes us from including the whole packages. At [25] you can find all the slides, documents, videos and labs used in the course. The course starts with an introduction to the most recent trends in computer architecture, and then it explores advanced processor design (deep pipelining, multi-cycle operations, speculative execution and branch prediction), the memory system (memory hierarchy, scratchpad memories), the I/O system, benchmarking and SoC design. The course includes 5 modules and 9 labs (see Table 3), distributed across 30 1.5-hour lectures plus 15 weekly 2-hour lab sessions.

In the labs, we use both the RVfpga and RVfpga-SoC packages, as well as the Ripes simulator [26], a visual computer architecture simulator and assembly code editor built for the RISC-V ISA. Each student is given a Nexys A7 FPGA board for the whole semester. The VeeR EH1 core fits on this board and is a more advanced processor than EL2, thus we mainly use EH1 as it allows us to explore concepts such as deep pipelining, superscalar execution, etc.

Before the first lab session, students install the provided virtual machine, which includes all the tools installed and ready to use. Although using the virtual machine is the most convenient approach, as it avoids installation problems or difficulties associated with the use of different operating systems, students may also install and use the software tools natively on their computers following the steps described in the Getting Started Guide.

In Module 1, students review the RISC-V ISA and complete Lab 1, where they learn to execute programs on the Nexys A7 board and the Whisper simulator. They then complete Lab 4, implementing an RGB-to-grayscale image processing algorithm in C and assembly. Module 2 introduces the VeeR EH1 processor, building on prior knowledge of a 5-stage core [19]. Students analyze its microarchitecture, including the 9-stage pipeline and superscalar execution. Labs 2–4 use the Ripes and RVfpga simulators to study hazards, performance counters (using Western Digital's Processor Support Package (PSP) [27]), and instruction execution. Module 3 covers memory hierarchy. In Lab 5, students explore data caches using Ripes and instruction caches using RVfpga. Lab 6 focuses on scratchpad memories (ICCM/DCCM) and benchmarks (Coremark, Dhrystone) to compare performance. Module 4 focuses on I/O systems. Lab 7 includes basic peripheral programming exercises, while Lab 8 guides students in extending the SoC by adding new peripherals in SystemVerilog. Finally, in Module 5, students use RVfpga-SoC to build a custom SoC from scratch with FuseSoC and run the Zephyr RTOS.

Around 85% of the students that attended the lab sessions were able to complete all or most of the labs, with an average grade of 6.4/10. According to the marks obtained and to their opinions in the final survey, the labs were a valuable resource for complementing and reinforcing the theoretical concepts. These are some of their comments:

- "The course, and particularly the lab sessions, provides a thorough understanding of computer architecture."
- "The lab activities enabled us to apply the concepts discussed in class and gain practical insights into how a commercial processor functions."
- "Initially, adapting to this new learning approach was quite challenging. However, once we developed the necessary skills, we started to truly enjoy the subject, achieving a clear understanding of how the core, cache, I/O, and other components operate. In my view, this is the ideal way to learn a subject of this nature."

2) COMPUTER ORGANIZATION AT UCM

This is a second-year course in the Software Engineering bachelor's degree program offered at UCM. Traditionally, this has been an entirely theoretical subject at UCM, leaving students without the opportunity to use or apply the concepts in practice. However, as with many fields, in the field of computing, many concepts can only be understood when they are both studied theoretically and practiced experimentally. Therefore, last year we decided to include 10 hours of lab sessions, based on RVfpga, to complement and reinforce theoretical concepts. In [25] you can find all the slides, documents videos and labs used on the course.

Compared to the Integrated Systems Architecture course discussed earlier, this course requires a more functional description with fewer details on certain aspects, as students have less background knowledge, less time for the labs, and the course has a higher enrolment. At this point, they have only studied Computer Fundamentals, where they only begin to learn the fundamentals of Digital Design and RISC-V Computer Architecture, so the exercises selected cannot be too advanced. Also, because enrolment is higher and we do not have enough FPGA boards for all students, the exercises must be completed in simulation only (specifically, using Whisper, RVfpga-ViDBo, Ripes and RVfpga-Pipeline).

The theoretical aspect of the course starts with a detailed review of those concepts introduced in their first year: RISC-V architecture and assembly programming, and the single/multi-cycle and pipelined processors from [19]. Then, the VeeR EH1 pipelined processor is used to introduce some advanced microarchitectural techniques, such as deep pipelining, superscalar execution, and multicycle operations. Finally, a generic I/O system and memory system are analyzed in detail.

The course includes a subset of the labs, in some cases simplified, shown in Table 3 for the Integrated Systems Architecture course discussed earlier. For the labs, the students work in groups of four, again using the Virtual Machine installed on their own laptops. The first lab is a review of the RISC-V architecture and is analogous to Lab 1 in Table 3; however, this time everything must be executed in Whisper, and the exercises are simpler, mainly selected exercises from Lab 2 of RVfpga (see TABLE 1). Lab 2 is the same as Lab 2 in Table 3; it uses the Ripes and RVfpga-Pipeline simulators to analyze microarchitectural techniques, such as structural and data hazards, branch instructions and associated control hazards (examined with different branch predictors), the operation of the Secondary ALU, and so on. Lab 3 is a simplified version of Lab 5 in Table 3; it analyzes the cache using the Ripes simulator. Finally, Lab 4 is the same as Lab 7 in Table 3, with the only difference that the exercises must be executed on RVfpga-ViDBo; this lab gives hands-on exercises about I/O on RVfpga. The students do not extend RVfpga's I/O system, because they do not yet have knowledge of HDL programming. After completing each lab, each student group must send a report that shows their exercise solutions accompanied by detailed explanations.

The results were quite good, as around 80% of the students were able to successfully complete all labs with an average grade of 7.7/10, and the final grades were higher than in previous years, where no labs were included. In addition, at the end of the semester, the students must complete a survey about the course, and they rated it with an average grade of 8.8/10, significantly above the average grade in UCM courses. Two student comments are:

- "The way the professor used the labs to clarify theoretical concepts should stand out as a model for other courses."
- "Personally, I'm not particularly passionate about Computer Architecture related subjects, I'm more into software. However, I must admit that this professor makes the subject enjoyable and the lab sessions clearly demonstrate the usefulness of the theoretical classes."

3) SoC DESIGN WITH PROGRAMMABLE LOGIC AT PSU

This course is part of a two-course sequence in the MSEE embedded systems track at Portland State University (PSU), focusing on FPGA-based SoC design. It uses RVfpga, the Catapult IDE for software development, and targets the Nexys A7 and Boolean FPGA boards. The course is project-based, with several two-week projects and a final team project (see Table 4). Grades are based on these projects and two exams. Students generally have experience in C and SystemVerilog, though some are new to RISC-V or large HDL designs. Code readability is part of the evaluation.

TABLE 4. Pojects and RVfpga resources.

Project	RVfpga Resources
Getting Started	RVfpga Getting Started Guide. Lab 2: RISC-V Assembly Language. Lab 3: Function Calls
Adding pushbuttons and RGB LED support to RVfpga	Lab 5: Creating a Vivado Project. Lab 6: Introduction to I/O. Lab 7: 7-segment Displays
Final Project	Lab 8: Timer. Lab 9: Interrupt-Driven I/O. Lab 10: Serial Buses



FIGURE 9. 4-function calculator.

Topics include FPGA architecture, synthesis, timing constraints, clock domain crossing, and the RISC-V ISA. A walkthrough of the RVfpga code helps students understand its structure. The short 10-week term limits deep architectural discussion. The first project uses the RVfpga Getting Started Guide to install tools and run a simple test, followed by a short design report. In the second project, students extend the RVfpga-EL2 SoC by adding pushbuttons and RGB LED control via custom peripherals using SystemVerilog and Vivado. They write two applications: one to display a counter and another to control PWM duty cycles shown on the 7-segment display. To manage scope, a base PWM module is provided for integration as a Wishbone peripheral. The third project adds VGA functionality. Students build a simple VGA controller and application to display and animate characters on screen, using a 25 MHz clock and HDMI output. Though basic, it enables richer final projects. One group expanded it into a 4-function calculator (Fig. 9).

The final project is team-based and student-proposed. Instructors provide feedback and adjust goals as needed. Evaluation is based on presentation, implementation, documentation, and code quality. A "Wall of Fame" is awarded to the best-executed project, such as a fully playable Tetris game developed in Winter 2023.

4) RISC-V HACKATHON AT THE RUPPIN ACADEMIC CENTERY

The RISC-V hackathon has been introduced as a new course for undergraduate students in the computer and electrical

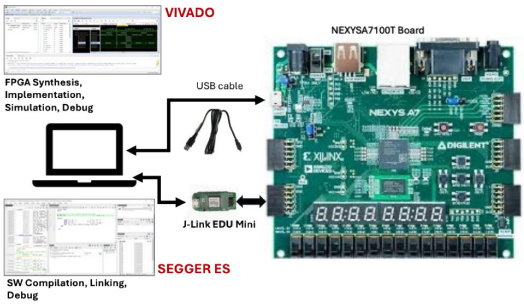


FIGURE 10. The hackathon development platform.

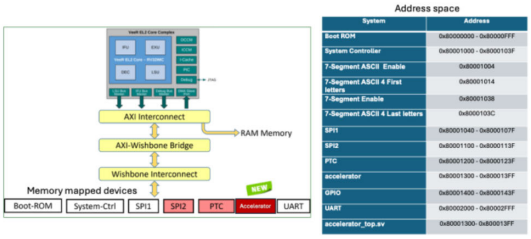


FIGURE 11. Memory-mapped accelerator in VeeR EL2 SoC.

engineering departments at the Ruppin Academic Center in Israel. This multidisciplinary challenge incorporates system-level technical skills, including microarchitecture, chip design, software development and machine learning. The VeeR EL2-based RVfpga SoC was used and implemented on the Nexys A7 FPGA board. Hardware development employed the Vivado Design Suite, while Segger Embedded Studio provided an integrated environment for software development, including compilation, simulation and debugging. Fig. 10 illustrates the complete development kit used in the hackathon.

Each student received a kit and attended two preparatory workshops held prior to the event. The first workshop focused on installing the platform and running programs in C on the RISC-V SoC. Emphasis was placed on the debugging capabilities offered by Segger (step-by-step execution, breakpoints, register/memory visibility), simulation of the SoC, and the use of Vivado's Integrated Logic Analyzer (iLA).

The second workshop focused on designing a memory-mapped accelerator integrated into the RVfpga platform using the Wishbone bus (Fig. 11). The accelerator included four memory-mapped registers: two for input data, one for the result, and one for control. It demonstrated parallel vector addition over 4-byte vectors. Students were then asked to extend it by implementing vector multiplication and increasing the vector width from 4 to 8 bytes.

At the main event, 30 students were challenged to accelerate a Fast Fourier Transform (FFT) algorithm implemented in fixed-point C over 16 samples. The goal was to achieve the highest speedup while minimizing area and power overhead, requiring a careful balance between performance and efficiency. After the event, students completed a questionnaire reflecting on their experience. Feedback highlighted the hackathon's strong contribution to their understanding of accelerator microarchitecture, system-level design, and hardware/software co-design involving programmable cores and custom accelerators.

TABLE 5. Course modules and corresponding laboratory sessions.

Module	Laboratory Session
Module 1: CPU Structure	Lab 1: Introduction to Ripes Simulator
Module 2: RISC-V ISA	Lab 2: Exercises with Ripes; Introduction to RVfpga project Lab 3: Loops and advanced features; Introduction to compilers Lab 4: Subroutines; More about compilers
Module 3: Introduction to RISC-V microarchitectures	Lab 5: Same ISA, different microprocessors
Module 4: Memory, access methods and memory hierarchy	Lab 6: Memory and memory hierarchy Lab 7: Cache memory; More about memory hierarchy
Module 5: IO Interfaces	Lab 8: Introduction to GPIOs Lab 9: Interrupts

5) IC AND CA COURSES AT UB

The Computer Engineering bachelor's degree at the University of Barcelona includes 3 courses in the field of Computer Architecture: Introduction to Computers (IC), Computer Architecture (CA), and Programming of Embedded Architectures (PAE). RVfpga is extensively used in the first two courses, alongside the RISC-V Ripes simulator. In the third course, the ESP32-C3 development board is used instead.

The IC course (second semester of the first year) builds upon two initial courses: Basic Digital Design (BDD), where students program state machines and ALUs in Verilog, and an introductory programming course (C/C++). IC focuses on processor components and the RISC-V single-cycle model, followed by the ISA using Ripes. Students are introduced to compilers, control units, memory systems, and basic I/O.

As shown in Table 5, each module is paired with lab sessions, completed using a virtual Ubuntu machine with Ripes and VSCode with PlatformIO, supporting RVfpga and the Nexys A7 board. Labs 3 and 4 include exercises such as writing recursive functions to study stack behavior and comparing handwritten and compiler-generated assembly. In Lab 5, students explore execution time across different RISC-V microarchitectures using Ripes. Memory-related concepts are addressed in Labs 6 and 7, including access types and cache behavior. Though RVfpga is not directly used in this module, the acquired knowledge prepares students for the CA course. Module 5 introduces I/O interfaces and peripherals. Labs use RVfpga to interact with LEDs, 7-segment displays, and switches. Lab 9 covers interrupts with guided exercises due to its complexity.

The CA course is built on IC, with 26 lectures and 26 problem-solving sessions (see Table 6). Topics include pipelining, superscalar execution, memory hierarchies, and I/O systems. Students do not have prior HDL experience,

TABLE 6. Modules in CA and lectures related to RVfpga.

Module	Laboratory Sessions
Module 1: Processor optimization	RVfpga Labs 14 to 17
Module 2: Memory Hierarchy	RVfpga Lab 19
Module 3: Memory Management	RVfpga Lab 20
Module 4: IO Interfaces and Buses	RVfpga Labs 6 and 7

so lab material from RVfpga is presented by instructors in master-class format. RVfpga Labs 11–17 support Module 1 on processor optimization, especially hazards (Labs 14–17). Lab 19 is used in Module 2 to study instruction caches, and Lab 20 in Module 3 to explore low-latency scratchpad memories (ICCM/DCCM), which include configurable ECC protection. In Module 4, Labs 6 and 7 reinforce I/O concepts, again taught in guided sessions due to the HDL components.

B. BACHELOR/MASTER'S DEGREE PROJECTS

In addition to the bachelor/master's degree courses described in Section A, RVfpga is also an excellent resource to use in final projects related to computer architecture. We next summarize some projects that have been developed and successfully completed over the last few years.

1) FPGA IMPLEMENTATION OF AN AD-HOC RISC-V SYSTEM-ON-CHIP FOR INDUSTRIAL IoT

In July 2020, Daniel León González, an IoT master's degree student at UCM, completed his master's degree thesis [28]. The project addresses the hardware implementation, using an Artix-7 FPGA, of a prototype IoT node based on RVfpga. It presents the implemented custom SoC and the development of the necessary Zephyr Operating System (OS) drivers to support a proof-of-concept application, which is deployed in a star network around a custom border router. End-to-end messages can be sent and received between the node and the ThingSpeak cloud platform. The thesis includes an analysis of the existing RISC-V processor implementations, a description of the required elements, and a guide to environment configuration and steps to build the project.

2) FLOATING-POINT EXTENSIONS FOR THE SweRV EH1 CORE

In July 2023, Alejandro Perea RodrÍguez, a student at UCM, completed his Computer Engineering bachelor's degree thesis based on RVfpga [17]. The project is aimed at adding the FPnew floating point unit [29] in the RVfpga SoC. After completion of the processor Zfinx extension, a series of verification tests were performed to check the correct operation of the new unit. The code developed in this project can be found at [30].

3) EXTENSION OF THE SIMULATION TOOLS FOR RVfpga

In February 2024, Óscar Lobato Parra, a student at UCM, completed his Computer Science bachelor's degree thesis based on RVfpga [31]. The project mainly extends the functionality of the RVfpga-ViDBo simulator by adding and verifying a tri-color LED. This device helps carry out in simulation the advanced practical exercises from Lab 8, where the PTC peripheral is used to generate a PWM signal that controls the RGB color shown by the tri-color LED.

4) HUB FORMAT FOR FLOATING-POINT FOR RVfpga

In July 2024, Alfonso MartÍnez Conejo, a student at the University of Malaga, completed his Computer Engineering thesis based on RVfpga [32]. In this work, the floating-point addition and multiplication algorithms proposed in Lab 18 are adapted to the new HUB format through modifying the VeeR EH1 core. HUB is an emerging format that simplifies hardware and reduces execution time for implementations that require rounding to nearest. Its key feature is that rounding to nearest is achieved by truncation, eliminating the need for the hardware typically required for rounding in conventional formats (which involves a final addition with carry propagation). This reduction in hardware requirements results in shorter execution times, as demonstrated in this thesis.

5) PROJECTS BASED ON RVfpga AT THE UNIVERSITY OF SHEFFIELD

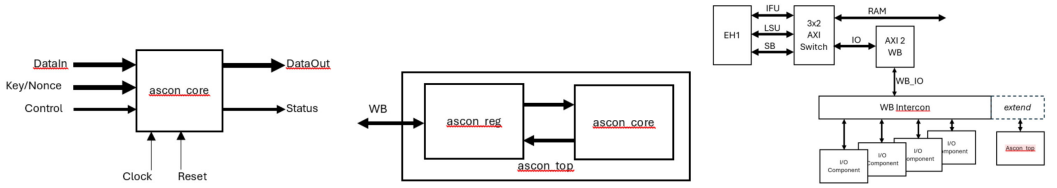
We were keen to use the resources from this project in the Department of Electrical and Electronic Engineering at the University of Sheffield to upskill students in HDL and SoC design. Since we were not ready to offer a full module, we introduced final-year undergraduate (BEng/MEng) and taught postgraduate (MSc) projects based on RVfpga, each occupying one-third of the academic year.

Up to 30 students may take part each year. They receive access to the IP and RVfpga Lab resources, with small group support and full-cohort meetings. Early stages focus on installing the tools, adapting the IP to their needs, and improving HDL skills. Later, students work independently on specific projects, with supervisors ensuring academic integrity where topics may overlap.

Projects often involve adapting the RVfpga SoC to add new functionality. A typical example from 2023–24 was integrating an ASCON lightweight cipher. Steps included:

1) The core design was simulated (in Vivado) to establish functionality (Fig. 12(a)).
2) Thereafter, it was wrapped in an appropriate memory-mapped/register-based interface allowing it to interface to the RISC-V processor (Fig. 12(b)).
3) The RVfpga design was then adapted (within Swervolf_core/wb_intercon) to implement the new component within the RVfpga SoC (Fig. 12(c)).
4) This design was also simulated using Verilator, running C code that controlled the device.
5) Finally, the overall design was synthesized, implemented, and tested on an FPGA.

Students are also asked to consider use cases, test strategy, and performance. Many face a steep learning curve, especially those with only limited Verilog experience,



(a) ASCON core (b) Wrapped system (c) New component of the RVfpga SoC

FIGURE 12. Integration of an ASCON lightweight cipher.

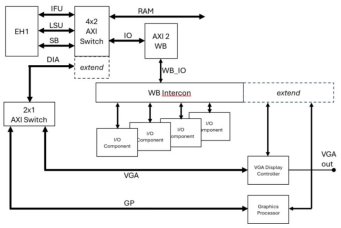


FIGURE 13. Overall design.

so strong supervisor support is essential. In fact, one lesson learned is that these projects are only feasible when supervisors are well-versed in the tools or have access to expert help. To manage scope, we now discourage final FPGA implementation. Projects often conclude after simulation; step 4 is a typical endpoint.

A more complex master's project involved a VGA display controller and a simple graphics processor (Fig. 13). Two new memory-mapped components were developed:

- VGA Display: Fetches pixel data from RAM and controls resolution, timing, and format.
- Graphics Processor: Issues drawing commands that update the RAM.

Both components access RAM via an extended AXI switch. Additional modifications included adapting axi2wb to support 64-bit writes. The system currently runs in simulation and is still under development.

6) EVALUATION OF REAL TIME OPERATING SYSTEM IN RISC-V

In 2024, Rui Ferreira, a student at the Instituto Superior Técnico (Technical University of Lisbon), completed his thesis based on RVfpga. In this thesis, the VeeRwolf SoC was modified to incorporate a hardware accelerator for matrix multiplication. The modifications were mainly to the AXI-4 interconnect to provide an additional AXI-4 master port to control the hardware accelerator.

Initially, three scenarios were proposed for managing the hardware accelerator using Zephyr RTOS mechanisms that enable the RISC-V processor to execute different tasks, while the accelerator works in parallel. In the first one, a single thread accesses the accelerator. The second scenario has several threads competing to access the accelerator and semaphores control access attempts. The third one uses a dedicated thread for managing the accelerator's access. Computational threads send requests to the manager thread to use the accelerator.

To evaluate these scenarios, two use case applications were developed and executed on the Nexys A7 FPGA board: matrix multiplication and a neural network to predict the digits of the MNIST dataset. The execution times of the different scenarios show that using a manager thread is more time-efficient than multiple threads competing for access to the accelerator, while also offering greater control over which thread accesses the accelerator.

7) FAULT-TOLERANT RISC-V PROCESSOR FOR SPACE APPLICATIONS

In year 2024, Jaime Aguiar, a student at the Instituto Superior Técnico (Technical University of Lisbon), completed his thesis based on RVfpga. This thesis explores the implementation of a Fault-Tolerant RISC-V system for space applications. The VeeR EL2-based RVfpga SoC was thoroughly studied, and its implementation simplified. This effort was intended to reduce the EL2 system's complexity and facilitate modifications for doing research on fault-tolerant methods alternative to triple-modular redundancy. Various fault tolerance mechanisms, including one already implemented in the CPU, were examined through simulations and error injection via ICAP.

One of the main advantages of using VeeR EL2 is the fact that its Verilog code follows a modular structure in which even the CPU registers are instantiated as a component.

Even though the research was done on an oversimplified version of the VeeR EL2, it is possible to extend the fault-tolerance mechanisms to apply them to the original VeeR EL2.

C. EdX COURSE

In 2023, we released a MOOC based on RVfpga [10]. This MOOC was developed in edX and it includes 10 chapters that cover labs 1-4, 6-9, and 11 of the RVfpga course. The course begins in Chapter 1 with an overview of the course itself, the RVfpga system, and the tools, including detailed instructions with videos that show how to install and use the tools. The remaining nine chapters (2-10) cover nine RVfpga labs, one per chapter: labs 1 to 4, about RISC-V assembly programming, and labs 6 to 9, about I/O, and Lab 11, an in-depth discussion of the VeeR EH1 core. These labs are called chapters 2-10 in the edX course. Each chapter

includes detailed instructions, hands-on tutorials, and videos describing the principles and theoretical foundation of the topic covered in the chapter and showing demonstrations of how to use and practice these principles. Each chapter also includes multiple choice questions and exercises for practicing and gaining hands-on experience with the topics taught in the chapter.

Like the full RVfpga course, the edX course provides the entire source code (Verilog and SystemVerilog) for the RVfpga SoC. Course participants may use, modify, and execute this source code throughout the course, using the RVfpga simulation tools: RVfpga-ViDBo, RVfpga-Trace, RVfpga-Pipeline, and RVfpga-Whisper. Because the course may be completed in simulation only, the course can be completed without cost. Optionally, participants may also use the Nexys A7 FPGA board to test their modified RVfpga system and code that they generate throughout the chapters.

D. MICROCREDENTIAL BASED ON RVFGA

Microcredentials are small, targeted educational courses that allow users to certify that they have learned a specific skill or subject area. This kind of course is highly supported and recommended by the European Union [33].

In 2025, we offered a new 60-hour microcredential at UCM called ''RISC-V: Architecture and design based on VeeR commercial cores on FPGA'', which included 250 hours of training activities based on the RVfpga course in Computer Architecture and the RVfpga-SoC course in SoC design. Some important companies developing RISC-V processors have shown interest in the content of this course and in hiring the students who took it. All materials can be accessed at [25].

This microcredential contains 4 modules that follow a similar organization as RVfpga. The course begins with an extensive introduction to the RISC-V architecture and assembly programming, based on RVfpga labs 1-4, and continues with a series of input/output practices using the RVfpga SoC, based on RVfpga labs 5-10. Subsequently, a detailed study of the VeeR cores is carried out, using RVfpga labs 11-20. Finally, the deployment of a complete application is carried out, using a Zephyr Project as an RTOS as well as TensorFlow Lite for microcontrollers, based on the RVfpga-SoC labs.

E. WORKSHOPS, TUTORIALS, WEBINARS, AND USE IN RESEARCH

To increase accessibility to this RVfpga course, we have run several international ''Train-the-Teacher'' workshops and tutorials since May 2022, five in the U.S., six in Europe (the workshop from London was recorded and is available in [34]), one in Israel, two in Japan, one in Taiwan, and multiple events across China. With guidance from the authors, these hands-on events enabled attendees to quickly run and use the RISC-V system and its tools.

Although these are not the usual teaching activities, in the sense that they are not instructed in the context of bachelor/master's degree courses, and these workshops are not aimed at students but instead at professors. These 1-day intensive practical workshops describe the main contents of the RVfpga course and guide students through subsets of select labs. Besides, these events have been a great method of obtaining valuable feedback from experts in the field who are able to test the materials in place.

The workshop includes the most important aspects of RVfpga, describing the system, showing attendees how to use and write code for it, and using hands-on practice to explore the peripherals and other features highlighted in the labs, including interrupts, performance counters, and benchmarking. Within the first hour of the workshop, users are able to write, compile, and run RISC-V programs on a commercial RISC-V core and SoC running in both hardware and simulation.

For the hands-on activities, we lend the attendees a Nexys A7 board (one board per student) where they can run the RVfpga SoC and complete all the labs. As for the RVfpga tools, we provide a Virtual Machine with everything already installed, that the attendees have to install and test the Virtual Machine on their laptops before the day of the workshop. In our experience after several workshops, this is the best approach, as it avoids many installation problems and saves a lot of time. Responses from attendees include the following:

- ''Excellent demonstration how real hardware and emulation matches, nice integration and IPC [instructions per cycle] and performance counter experiments.''
- ''I enjoyed the practical knowledge about RVfpga. My first successful FPGA project!''
- ''A clear, focused, comprehensive [workshop] linking hardware, software, and RISC-V and relevant computer architecture within a workable toolflow and affordable hardware options.''

In addition to the workshops, the authors hosted a Webinar sponsored by DigiKey to introduce RVfpga where 750 attended live, and more than 1,000 additional viewers subsequently watched the presentation [35]. Finally, let us mention that some researchers have also used RVfpga as their experimental infrastructure ([36], [37], [38]).

IV. RELATED WORK

In the latest years, numerous educational resources have been developed to help users deepen their understanding of the RISC-V architecture and its applications. Fortunately, RISC-V International seeks to coordinate efforts by centralizing RISC-V teaching resources in a recently-launched repository [39], where a community-driven compilation of RISC-V resources and learning material is provided. The contents are continuously updated by the community and categorized based on the RISC-V scope and expertise level (from beginner to intermediate level). We should highlight that our edX RVfpga course is one of the recommended resources in the introduction (''After studying the Digital Design book in this section [beginner-level], you could jump

to the intermediate-level edx RVfpga course if you wish as it expands on concepts discussed in the book.'').

Several RISC-V learning materials, both theoretical and practical, are recommended in this repository [39]. As for the purely theoretical resources, the most relevant books on RISC-V computer architecture are included as a reference: Harris and Harris [19], Patterson and Waterman [40], and Patterson and Hennessy [20]. As for the practical resources, in addition to RVfpga (both the edX course [10] and the extended version [1] are referenced), other courses detailed next are also included in the repository.

The ''Computer Architecture Basics'' course [41] provides lectures, slides and videos about different topics such as arithmetic, CPU design, memory hierarchy, pipelined/speculative execution and I/O systems, as well as nine tutorials and five labs based on the QtRvSim simulator. The tutorials describe the most important computer architecture concepts, and how to connect to an Avnet MicroZed board. Accompanying homework exercises delve into the concepts examined in the lectures and tutorials: number representation, image processing, processor pipelines, and code analysis. This beginner-level course has some weaknesses, as it is mainly oriented to simulation, and peripheral support is very limited; it also uses an outdated bus (PCI) and does not address important architecture aspects such as superscalar execution, interrupt-based I/O or scratchpad memories. In addition, some materials are only available for students and in some cases the course has not yet been ported to RISC-V and uses the MIPS architecture as it did in older versions.

Several resources included in the repository [39] show how to design and build a RISC-V processor of varying complexity and performance. One of those resources is learn-fpga [42], which provides two comprehensive courses. The first one (beginner-level) starts from a very basic Verilog module that makes a LED blink and morphs it step by step into a basic yet fully functional RISC-V SoC implemented on an FPGA. It also explains how to write programs in C and assembly for the RISC-V SoC. The second course (intermediate-level) explains how to transform the basic softcore from the first course into an efficient pipelined processor. As [41], these are very interesting and comprehensive materials with strengths but also some weaknesses. As opposed to [41], it is based on an FPGA and can be also run in simulation (using Icarus Verilog or Verilator). Besides, it uses synthesis and place and route freeware tools (Yosys and nextpnr), a clear advantage for users. In terms of its weaknesses, the course is based on a non-professional and non-verified CPU (FemtoRV); it does not provide exercises but only guided activities; and some topics are not covered (such as interrupts, which are under construction, or advanced microarchitectural techniques such as superscalar execution).

Another option in the same line as learn-fpga is Quick Silicon's ''RISC-V Processor Design'' [43], a beginner-level course that provides skills and hands-on experience to the user to understand and design RISC-V-based processors from scratch. It provides several tutorials and videos that analyze fundamental concepts of computer architecture and processor design, the unprivileged RISC-V ISA, how to design the single-cycle RV32I-compliant YARP (Yet Another RISC-V Processor) core from scratch using SystemVerilog, and how to create RISC-V assembly programs and execute them on the designed processor. The course's weaknesses are a lack of important microarchitectural and SoC concepts, using a non-verified CPU and, as opposed to most of the other resources in the repository [39], this course costs $60.

A different beginner-level option is [44], an edX course that shows how to create a RISC-V CPU with modern open-source circuit design tools (Makerchip online IDE), methodologies and microarchitecture, all from a web browser. As with other edX courses, this course is free but can be optionally upgraded to be able to access graded assignments required for the certificate. The course does not use conventional HDL (such as Verilog or VHDL) but a specific language called Transaction-Level Verilog. It also relies on simulation only, and the CPU is simple and unverified. A similar course, which is not referenced in the repository, is [45], where the authors propose the creation of a pipelined RISC-V microarchitecture model using the same language and IDE as [44]: the TL-Verilog hardware description language and the Makerchip online development environment. The course includes 11 labs about the following topics: RISC-V architecture and assembly language; design in TL-Verilog; RISC-V microarchitecture: pipelining, branches; memory access; and verification.

Another beginner-level course also mentioned in the repository is [46], which also builds a general-purpose computer and uses it on a simulator and a CPU emulator; however, as opposed to previous resources, this course does not use RISC-V but a symbolic language called Hack.

An intermediate-level resource provided in the repository [39] is Shakti [47], an open-source initiative which provides a complete development ecosystem where several processors, software tools, documentation and assignments are included. It provides an installation guide plus four assignments on RISC-V assembly programming, exceptions and interrupts, privilege levels, and paging and PMP. The main strength of this course is the variety of open-source processors provided. However, the course uses an Arty-7 100t FPGA board, which costs around $300, it only works in Linux (Ubuntu 18.04 is the verified distribution), and it does not cover many important computer architecture topics such as microarchitecture or memory hierarchy.

It is also worth noting that RISC-V International, in association with the Linux Foundation, recently announced a certification [39] where the student is guided in the process of learning RISC-V architecture and then complete an evaluation which demonstrates if the individual possesses the fundamental, entry-level knowledge and skills required of RISC-V hardware and software professionals. RISC-V

International recommends various RISC-V Training Partners courses and RISC-V edX courses. Several courses, in addition to RVfpga, are included; however most of them require payment. Vicilogic [48] is an online RISC-V learning, assessment, prototyping, and course builder platform that uses remote FPGA hardware. Silicon [49] provides a set of five RISC-V courses, three of which are free and explore the architecture of RISC-V ISA and RV32I RTL design, the RISC-V RV32I RTL design using Verilog HDL, and that validate RISC-V RV32I RTL designs effectively with an UVM-based verification course. Finally, VSD-RISCV [50] offers courses that analyze the RISC-V architecture ($10 per course).

For the RISC-V edX courses recommended for the certificate [39], several resources are provided, most of which are free of charge. The three fundamental RISC-V courses [51], [52], [53] mainly analyze the RISC-V ISA and RISC-V assembly programming, as well as RISC-V tools, the GNU C compiler for RISC-V, and some basic concepts on OSs and libraries. In addition, some advanced edX courses are also listed, which analyze the microarchitecture of RISC-V CPUs [10], [44], embedded Linux, FreeRTOS, and Bare-Metal application development [54], [55], [56], [57], [58], [59].

Finally, although not included in the previous repositories, various RISC-V platforms have been introduced. In [60], the objective is to train students in hardware-software co-design of integrated systems. The course uses the Rocket Chip design platform, which allows students to begin with fast prototyping on a physical implementation (the Nexys A7 FPGA board). A demo is provided where students select an image and, optionally, a filter, using pushbuttons. Then, several labs are proposed on the following topics: getting started with the hardware and software environment; exceptions, traps and interrupts; multi-tasking, multi-processing, memory coherence and caches; and adding custom hardware peripherals. Finally, a project is proposed where the students are asked to develop the software to reproduce the behavior of the demo. These are interesting and comprehensive resources. The main problem is that the platform has not been updated for more than three years. In [61] the authors provide an open-source RISC-V platform for education and research, which includes a configurable RV32IMC core and the MicroRV32 SoC, that, in addition to the core, includes several peripherals such as a UART, GPIO, an interrupt controller, and memory. They can be run in both simulation and on Lattice Semiconductor iCE40 FPGAs. The core has been verified via testbenches, software-based unit tests, example software applications, and formal verification, a cross-level approach in which the RTL and Virtual Prototype (VP) models are compared in a co-simulation setting. The BRISC-V toolbox [62] is the Boston RISC-V architecture design exploration suite. BRISC-V is comprised of several processor architectures, a simulator, and a visual Verilog file generation tool, for education and research projects. The Trireme Platform [63] includes everything you need to bring up the hardware and software of a custom RISC-V system, from ultra-low-power microcontrollers to high-performance multi-core processors.

Although RVfpga still has room for improvement, it covers most of the gaps and content not included in the previously analyzed resources, such as advanced microarchitecture (superscalar execution, high-performance branch prediction, cache and scratchpad, benchmarking, interrupt-based I/O), support for low-cost FPGA boards (Basys 3 or Boolean), support for various simulation options (pipeline viewer, virtual FPGA board), use of commercial and verified cores (VeeR EH1 and EL2), etc. RVfpga is comprehensive yet modular and flexible, making it easy to adapt to different levels and profiles.

V. CONCLUSION AND FUTURE WORK

In this paper we described the two RVfpga courses we developed on computer architecture and SoC design. These courses describe how to use the RISC-V toolchain, how to program a commercial RISC-V SoC in C and assembly, how to target the SoC to various simulators or, optionally, to an FPGA board, how to use and extend the SoC's peripherals, how to understand and modify the core microarchitecture, and how to build the RVfpga SoC from scratch and run an RTOS on it. We have also examined and discussed several experiences using the RVfpga packages, and we have analyzed the latest computing curriculum guidelines developed by key organizations such as ACM and IEEE, justifying how the RVfpga courses align with these guidelines. Finally, we have conducted an in-depth examination of related work on RISC-V education and compared it with our own work.

As for our future work, we plan on making several improvements to the RVfpga course. In the short-term, we plan to release v3.1 that improves some parts of the current v3.0 RVfpga course in computer architecture. In the medium/long-term, we plan to release a v4.0, where additional extensions will be included, such as new labs about buses, exceptions, new peripherals (VGA, Ethernet...), and verification. We would also like to extend the RVfpga-Pipeline simulator, by integrating the VeeR EH1/EL2 cores in the Ripes simulator, following the instructions from [26]. Finally, we also plan to review, improve, and extend the current RVfpga-SoC course.

REFERENCES

[1] Imagination University Programme. (2023). *RVfpga V3.0 Teaching Materials*. [Online]. Available: https://university.imgtec.com/rvfpga-el2-v3-0-english-downloads-page/

[2] Imagination University Programme. (2023). *RVfpga—Introduction to RVfpgaSoC*. [Online]. Available: https://university.imgtec.com/rvfpgasoc-download-page-en/

[3] (Dec. 2022). *VeeR EH1 RISC-V Core*. [Online]. Available: https://github.com/chipsalliance/Cores-VeeR-EH1

[4] ChipsAlliance. (Dec. 2022). *VeeR EL2 RISC-V Core*. [Online]. Available: https://github.com/chipsalliance/Cores-VeeR-EL2

[5] *Basys 3 Artix-7 FPGA Trainer Board*. Accessed: Aug. 22, 2025. [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/start

[6] *Boolean Board*. Accessed: Aug. 22, 2025. [Online]. Available: https://www.realdigital.org/hardware/boolean

[7] *Nexys A7 Artix-7 FPGA*. Accessed: Aug. 22, 2025. [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/start
[8] S. L. Harris, D. Chaver, L. Piñuel, J. I. Gomez-Perez, M. H. Liaqat, Z. L. Kakakhel, O. Kindgren, and R. Owen, "RVfpga: Using a RISC-V core targeted to an FPGA in computer architecture education," in *Proc. 31st Int. Conf. Field-Program. Log. Appl. (FPL)*, Aug. 2021, pp. 145–150.
[9] S. Harris, D. Chaver, L. Piñuel, O. Kindgren, and R. Owen, "RVfpga: Computer architecture course and MOOC using a RISC-V SoC targeted to an FPGA and simulation," in *Proc. ASEE Annu. Conf. Expo.*, 2023, pp. 1–12.
[10] *Computer Architecture With an Industrial RISC-V Core [RVfpga]*. Accessed: Sep. 1, 2025. [Online]. Available: https://www.edx.org/es/learn/computer-programming/the-linux-foundation-computer-architecture-with-an-industrial-risc-v-core
[11] O. Kindgren. *VeeRwolf*. Accessed: Aug. 25, 2025. [Online]. Available: https://github.com/chipsalliance/VeeRwolf
[12] *RVfpga/EL2 Version 3.0 Videos*. Accessed: Aug. 27, 2025. [Online]. Available: https://www.youtube.com/playlist?list=PLnOXj03cuJjn7ecksWZEwCe8LVR-PDgud
[13] D. Chaver. (2025). *RVfpga Videos*. [Online]. Available: https://www.youtube.com/@RVfpgaVideos
[14] (2022). *RVfpga One-Day Workshop 2022*. [Online]. Available: https://youtube.com/playlist?list=PLnOXj03cuJjkes1OIMa8SFyn1yKQQY8aD
[15] *Verilator: The Fastest Verilog/SystemVerilog Simulator*. Accessed: Aug. 27, 2025. [Online]. Available: https://www.veripool.org/verilator/
[16] O. Kindgren. *Virtual Development Board*. Accessed: Aug. 27, 2025. [Online]. Available: https://github.com/olofk/vidbo
[17] A. Perea, A. del Barrio, and D. Chaver. (2023). *Extensiones De Punto Flotante Para El Core SweRV EH1*. Accessed: Aug. 27, 2025. [Online]. Available: https://docta.ucm.es/entities/publication/274f1b3a-c564-464d-8066-ef9aa4ae3c10
[18] *WD RISC-V Firmware Package*. Accessed: Aug. 27, 2025. [Online]. Available: https://github.com/chipsalliance/riscv-fw-infrastructure
[19] S. Harris and D. Harris, *Digital Design and Computer Architecture—RISC-V Edition*. San Mateo, CA, USA: Morgan Kaufmann, 2021.
[20] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The HW/SW Interface*. Amsterdam, The Netherlands: Elsevier, 2017.
[21] O. Kindgren. *FuseSoC*. Accessed: Oct. 21, 2025. [Online]. Available: https://github.com/olofk/fusesoc
[22] *CS2023: Computer Science Curricula*. Accessed: Aug. 29, 2025. [Online]. Available: https://csed.acm.org/
[23] (2020). *Computing Curricula 2020: Paradigms for Global Computing Education*. Accessed: Aug. 29, 2025. [Online]. Available: https://www.acm.org/education/curricula-recommendations
[24] *Caliptra Open Source Root of Trust Project*. Accessed: Nov. 20, 2025. [Online]. Available: https://www.chipsalliance.org/news/chips-alliance-welcomes-the-caliptra-open-source-root-of-trust-project/
[25] *Teaching Experiences At UCM Based on RVfpga*. Accessed: Oct. 20, 2025. [Online]. Available: https://github.com/artecs-group/RVfpga-sim-addons
[26] M. B. Petersen. *Ripes Simulator*. Accessed: Oct. 20, 2025. [Online]. Available: https://github.com/mortbopet/Ripes
[27] *RISC-V Firmware Package*. Accessed: Nov. 20, 2025. [Online]. Available: https://github.com/chipsalliance/riscv-fw-infrastructure
[28] D. Leon, L. Piñuel, and D. Chaver. (2023). *FPGA Implementation of an AD-HOC RISC-V System-On-chip for Industrial IoT*. Accessed: Aug. 29, 2025. [Online]. Available: https://docta.ucm.es/entities/publication/9298fbe5-cabd-40a0-bec5-0501290c30d1
[29] *FPnew-New Floating-Point Unit With Transprecision Capabilities*. Accessed: Aug. 29, 2025. [Online]. Available: https://github.com/openhwgroup/cvfpu
[30] A. Perea. *Extensiones De Punto Flotante Para El Core SweRV EH1*. Accessed: Aug. 29, 2025. [Online]. Available: https://github.com/aperea01/TFG-SweRV-EH1-FP
[31] O. Lobato, K. Olcoz, and D. Chaver. (2024). *Ampliación De La Funcionalidad De Las Herramientas De Simulación De RVFPGA*. Accessed: Aug. 29, 2025. [Online]. Available: https://docta.ucm.es/entities/publication/ae3d83a6-453b-4fc9-a9d9-80f0a02291cb
[32] A. Martinez and J. Villalba. (2024). *Adding Functionalities to a RISC-V CPU Based on the RVFPGA Project*. Accessed: Oct. 1, 2025. [Online]. Available: https://hdl.handle.net/10630/34157
[33] *A European Approach to Micro-Credentials*. Accessed: Aug. 29, 2025. [Online]. Available: https://education.ec.europa.eu/education-levels/higher-education/micro-credentials
[34] (2022). *RVfpga One-Day Workshop-Imperial College London*. Accessed: Aug. 29, 2025. [Online]. Available: https://www.youtube.com/playlist?list=PLnOXj03cuJjkes1OIMa8SFyn1yKQQY8aD
[35] (Nov. 2022). *RVfpga Teaching Materials Webinar Hosted By @Digi-Key*. Accessed: Aug. 29, 2025. [Online]. Available: https://www.youtube.com/watch?v=ePv3xD3ZmnY
[36] J. C. V. Fernandes, H. da Rocha, and A. Espírito-Santo, "Encryption and authentication in smart transducers implemented in RISC-V softcore processors," in *Proc. IEEE Int. Conf. Ind. Technol. (ICIT)*, Apr. 2023, pp. 1–6.
[37] A. Murtaza, M. A. Pasha, S. Masud, M. Y. Qadri, and A. Basit, "FPGA based intelligent hardware trojan design and its SoC implementation," in *Proc. 30th IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, Dec. 2023, pp. 1–4.
[38] M. A. Ali, M. A. Pasha, S. Masud, and F. A. Ghazali, "FPGA design of a reconfigurable UART IP core and its SoC implementation for IoT applications," in *Proc. 25th Int. Multitopic Conf. (INMIC)*, Nov. 2023, pp. 1–6.
[39] *RISC-V Foundational Associate Certification*. Accessed: Aug. 26, 2025. [Online]. Available: https://riscv.org/certifications-and-courses/
[40] D. Patterson and A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*, 1st ed., Berkeley, CA, USA: Strawberry Canyon, 2017.
[41] P. Pisa. (Apr. 2024). *Computer Architecture Basics*. Accessed: Aug. 25, 2025. [Online]. Available: https://cw.fel.cvut.cz/wiki/courses/b35apo/en/lectures/start
[42] B. Levy. *Learn FPGA*. Accessed: Aug. 25, 2025. [Online]. Available: https://github.com/BrunoLevy/learn-fpga/tree/master
[43] *RISC-V Processor Design*. Accessed: Aug. 25, 2025. [Online]. Available: https://quicksilicon.in/course/riscv
[44] *Building a RISC-V CPU Core*. Accessed: Aug. 25, 2025. [Online]. Available: https://www.edx.org/learn/design/the-linux-foundation-building-a-risc-v-cpu-core
[45] P. Minev, V. Kukenska, I. Varbov, and M. Dinev, "A practical computer architecture education with RISC-V and TL-verilog," in *Proc. 32nd Int. Sci. Conf. Electron. (ET)*, Sep. 2023, pp. 1–6.
[46] S. Schocken and N. Nisan. *From Nand to Tetris*. Accessed: Aug. 25, 2025. [Online]. Available: https://www.nand2tetris.org/
[47] *Learn With SHAKTI*. Accessed: Aug. 25, 2025. [Online]. Available: https://shakti.org.in/learnwithshakti/intro.html
[48] *And2RISCV—RISC-V Online Tutor Course*. [Online]. Available: https://www.vicilogic.com/
[49] *RISC-V Online Courses*. Accessed: Sep. 1, 2025. [Online]. Available: https://elearn.maven-silicon.com/courses?categoriesids=7
[50] *VSD—RISCV: Instruction Set Architecture (ISA)*. Accessed: Sep. 1, 2025. [Online]. Available: https://www.udemy.com/course/vsd-riscv-instruction-set-architecture-isa-part-1a,https://www.udemy.com/course/vsd-riscv-instruction-set-architecture-isa-part-1b
[51] *RISC-V Fundamentals*. [Online]. Available: https://training.linuxfoundation.org/training/riscv-fundamentals-lfd210/
[52] *Introduction To RISC-V*. Accessed: Sep. 1, 2025. [Online]. Available: https://training.linuxfoundation.org/training/introduction-to-riscv-lfd110/
[53] *Foundations of RISC-V Assembly Programming*. Accessed: Sep. 1, 2025. [Online]. Available: https://training.linuxfoundation.org/training/foundations-of-risc-v-assembly-programming-lfd117x
[54] *Developing Embedded Linux Device Drivers*. Accessed: Sep. 1, 2025. [Online]. Available: https://training.linuxfoundation.org/training/developing-embedded-linux-device-drivers/
[55] The Linux Foundation. *Embedded Linux Development*. Accessed: Sep. 1, 2025. [Online]. Available: https://training.linuxfoundation.org/training/embedded-linux-development/
[56] The Linux Foundation. *Embedded Linux Platform Development With Yocto Project*. Accessed: Sep. 1, 2025. [Online]. Available: https://training.linuxfoundation.org/training/embedded-linux-development-with-yocto-project/
[57] The Linux Foundation. *Building Applications With RISC-V and FreeRTOS*. Accessed: Sep. 1, 2025. [Online]. Available: https://training.linuxfoundation.org/training/building-applications-with-risc-v-and-freertos-lfd112x

[58] The Linux Foundation. *Microcontroller Applications With RISC-V*. Accessed: Sep. 1, 2025. [Online]. Available: https://www.edx.org/learn/microcontrollers/the-linux-foundation-microcontroller-applications-with-risc-v

[59] *RISC-V Toolchain and Compiler Optimization Techniques*. [Online]. Available: https://www.edx.org/learn/computer-programming/the-linux-foundation-risc-v-toolchain-and-compiler-optimization-techniques

[60] *RISC-V Based SoC Platform for Research Development and Education*. Accessed: Aug. 25, 2025. [Online]. Available: https://tima-amfors.gricad-pages.univ-grenoble-alpes.fr/learnv/

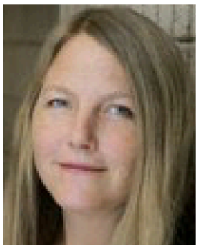
[61] S. Ahmadi-Pour, V. Herdt, and R. Drechsler, "MircoRV32: An open source RISC-V cross-level platform for education and research," in *Proc. Workshop Design Autom. CPS IoT*, May 2021, pp. 30–35.

[62] *BRISC-V Toolbox*. Accessed: Aug. 25, 2025. [Online]. Available: https://ascslab.org/research/briscv/index.html

[63] *Trireme RISC-V Design Platform—A Complete Platform for RISC-V Design Space Exploration*. Accessed: Aug. 25, 2025. [Online]. Available: https://www.trireme-riscv.org/

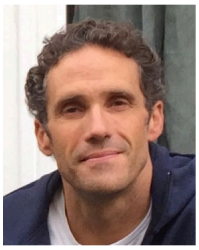


DANIEL CHAVER received the M.S. degree in physics from the University of Santiago de Compostela, Spain, in 1998, and the M.S. degree in electrical engineering and Ph.D. degree in computer science from the Complutense University of Madrid, Spain, in 2000 and 2006, respectively. He is currently an Associate Professor with the Department of Computer Architecture and Automation, Complutense University of Madrid. Since 2000, he has been actively involved in research and teaching, focusing on computer architecture.



SARAH HARRIS received the M.S. and Ph.D. degrees from Stanford University.

She is currently a Professor of electrical and computer engineering with the University of Nevada, Las Vegas. Before joining UNLV, in 2014, she was a Faculty Member with the Harvey Mudd College for ten years. She has also worked at HP, Nvidia, and the Technical University of Darmstadt and has collaborated with other companies, including Southwest Research Institute, Intel, and Imagination Technologies. She is the co-author of three popular textbooks, her most recent being *Digital Design and Computer Architecture: RISC-V Edition*, published in 2021. Her research interests include computer architecture and applications of embedded systems and machine learning to biomedical engineering and robotics.

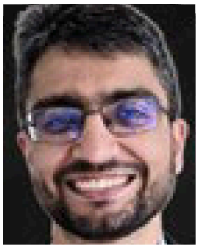


LUIS PINUEL received the M.Sc. and Ph.D. degrees in computer science from the Complutense University of Madrid (UCM), in 1996 and 2003, respectively.

He is currently an Associate Professor with the Department of Computer Architecture and Systems Engineering, UCM. He is the co-author of more than 70 publications in prestigious journals and international conferences, and several book chapters. He has advised or co-advised five Ph.D. dissertations. Worried about improving knowledge transfer between research institutions and industry, he has directed more than 15 research contracts with different companies, such as Texas Instruments, Indra, and Satlink. His research interests include computer architecture, high-performance computing, low-power microarchitectures, embedded systems, and resource management for emerging computing systems.



OLOF KINDGREN is an award-winning digital design engineer working for Qamcom Research and Technology. He is also a renowned open-source silicon expert involved in many projects both as an advocate and a developer to promote, assist, and advance open-source silicon within the industry, academia, and for hobbyists alike. On the advocacy, he is a Co-Founder and the Director of FOSSi Foundation, a contributor to RVFPGA, and serves as a RISC-V Ambassador. On the developer side, he leads several well-known open-source silicon projects, such as FuseSoC, Edalize, SERV, and VeeRwolf.



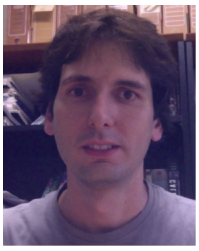
ZUBAIR KAKAKHEL brings a decade of software engineering experience (from working in an FTSE 500 company to a series A/B Startup). He founded AZKY Tech Laboratories, in 2020, and has grown to a team of 15 full-time staff and an amazing pool of highly experienced contractors all around the world. So far, he has helped more than 100 businesses of all sizes (one person Startups to SMEs and Enterprises) build and launch their software products.



CHRIS OWEN worked for Imagination Technologies for 21 years in a number of embedded software roles, most latterly leading the team developing the Catapult SDK RISC-V software development kit, and the Catapult Studio IDE and Visual Studio Code extension. Since leaving Imagination in November 2024, he has worked for Cotronix as a Contractor in the embedded software field.



ROY KRAVITZ received the B.S. degree in computer engineering and the M.S. degree in electrical engineering from the University of Illinois, Urbana. He is an Emeritus Instructor. He is currently teaching for the ECE Department, Portland State University. He has created and co-taught courses in FPGA design, digital systems design, embedded systems, and computer architecture for more than 25 years. His professional career spans 35 years in multiple companies and roles. He started as a Microprocessor Designer with Intel and managed engineering teams and projects at RadiSys and Serveron. His research interests include embedded system hardware and software, Android programming, digital system design using programmable logic, and computer architecture.



JOSE I. GOMEZ-PEREZ received the M.Sc. degree in computer science and the Ph.D. degree from the Complutense University of Madrid, Spain, in 2001 and 2007, respectively.

He is currently an Associate Professor with the Department of Computer Architecture and Automation, UCM, within the ArTeCS Group. During his Ph.D., he was a Visiting Researcher with Imec, Leuven, working on optimizing low-power embedded systems at both the compiler and system level. After his Ph.D., he moved to GPGPU computing, still at the compiler level. His current research interests include low-power embedded systems, focusing on the architectural impact of new resistive memory technologies, in the IoT ecosystems.



FERNANDO CASTRO received the M.S. degree in physics from the University of Santiago de Compostela, in 2000, and the M.S. degree in electrical engineering and the Ph.D. degree in computer science from the Complutense University of Madrid, in 2004 and 2008, respectively.

He is currently an Associate Professor with the Computer Architecture and Automation Department, Complutense University of Madrid. Since 2003, he continuously participated in competitive projects related to the computer architecture field. His research interests include energy-aware processor design, efficient memory management (including emerging non-volatile memory technologies), and OS scheduling on heterogeneous multiprocessors.



KATZALIN OLCOZ (Member, IEEE) is currently an Associate Professor with the Department of Computer Architecture and Automation, Complutense University of Madrid.

She is the Co-Leader of the Research Group ArTeCS and a member of the Spanish Red-RISCV. Her current research interests include high-performance and energy-aware computing, heterogeneous systems, memory hierarchy, and resource management for emerging computing systems. She has been in the program committee of top-tier IEEE and ACM conferences, including the ACM International Conference on Supercomputing and the RISC-V Summit Europe. She also acts as an Associate Editor of IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING and IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS.



JULIO VILLALBA-MORENO received the B.S. degree in physics from the University of Granada, in 1986, and the Ph.D. degree in computer science from the University of Málaga, Spain, in 1995.

From mid-1986 to 1991, he was a Design Engineer with the Research and Development Department, Fujitsu Spain. Since 2007, he has been a Full Professor with the Department of Computer Architecture, University of Málaga. He has been a member of the Steering Committee of the IEEE Symposium on Computer Arithmetic, since 2015, and served as the General Co-Chair of the 31st IEEE International Symposium on Computer Arithmetic (ARITH 2024).



ALEXANDER GRINSHPUN received the B.Sc. degree in electrical engineering. He is currently pursuing the Ph.D. degree with The Hebrew University of Jerusalem, Research "Exploring Asymmetric Transistor Aging in FPGAs and Mitigation Techniques for Enhanced Reliability".

From 1992 to 2016, he was with Intel Corporation (IDC), Haifa, Israel. He has more than 20 years of experience with Emulation, FPGA, ASIC design, and verification. Since 2016, he has worked on the Academia Ruppin Academic Center, Technion–Israel Institute of Technology, and The Hebrew University of Jerusalem.



FREDDY GABBAY (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from the Technion–Israel Institute of Technology.

He is currently the Head of the VLSI Systems Laboratory, Hebrew University. His professional career spans more than two decades of leadership roles in both academia and industry as a Researcher at Intel's Microprocessor Research Laboratory, a Senior Design Manager with Freescale Semiconductor, and the Vice President of Chip Design, Mellanox (Aquired by Nvidia). He holds 19 patents. His research interests include VLSI design, microelectronics, computer architecture, machine learning, and domain-specific accelerators.

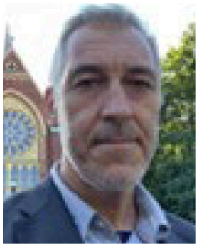


LUKE SEED (Member, IEEE) received the B.Eng. and Ph.D. degrees in electronic engineering from the Department of Electronic and Electrical Engineering, The University of Sheffield, in 1983 and 1989, respectively.

He joined the Academic Staff with the Department of Electronic and Electrical Engineering, in 1986, becoming a Senior Lecturer, in 1997. Over the years, his research has ranged from image processing, through computer vision, to low-power VLSI and 3D packaging technologies. His work, since 2002, has been focused on using numerically-generated holograms within photolithographic processes for applications, such as electronic packaging and non-planar antennas. He has been the Departmental Director of Learning and Teaching, since 2012, and Deputy Head of Department, since 2014.



RUI DUARTE (Senior Member, IEEE) is currently a Lecturer and a Researcher on computer architectures, FPGAs, and embedded systems (IoT). He looks forward to establishing collaborations with the (space) industry for research and development projects, training, and consultancy. His top skills are reconfigurable computing, computer architecture, fault-tolerant systems, field-programmable gate arrays (FPGA), and the Internet of Things (IoT).



MANUEL LÓPEZ has been an Associate Professor with UB, since 2007. His design and development research activity is devoted to the real time applications, system electronics designs, embedded software, low-power computer architectures, and communications. In these fields, he has mainly worked in the field of sensor networks and embedded systems. He is an expert in CAD tools for the design of PCB systems, such as Mentor Expedition PCB, Design Spark, Eagle, and KiCad. He participated in the Spanish section of the E.S.A. Solar Orbiter for the development of highly integrated PCB design. He was the Co-IP of the Project ''Development of new detectors for future colliders in particles physics.''



ÓSCAR ALONSO was born in Barcelona, in 1982. He received the Ph.D. degree in engineering and advanced technologies from the University of Barcelona, in 2012.

He is currently an Associate Professor with the University of Barcelona. His research focuses on VLSI circuit design, with experience in ASIC development for a wide range of applications. His include analog IPs for high-energy physics experiments, IPs for RISC-V-based processors, medical and biomedical devices, and SoC designs for autonomous and sustainable electronics. His recent work explores low-power and flexible IGZO-based circuits for environmentally conscious applications, including disposable and energy-harvesting IoT systems.



ROBERT OWEN created and directed three of the world's leading industry-to-university programs for Texas Instruments, Imagination Technologies, and RS Components. In parallel, he created and ran a successful media business. In 2019, he returned to Imagination to revive and reposition the Imagination University Program (IUP), which is now focused on the ''silicon engines'' of GPUs, NNAs, and new developments of a RISC-V-based family of CPUs. The application of these processor engines is driving strong needs for new education projects with a particular emphasis required for hands-on learning using industrially-proven technologies. His projects enable teachers of the next generation of engineers and programmers to understand how to solve real-world problems with these technologies. After many years of Industry-to-Academia collaboration, he has unique insight into this market and a great network of relationships around the world.